PORTFOLIO

Software Defined Networking Practicum

Naam student

Michyl Stuurop

Teamgenoot (opdracht 4)

Maarten Meindertsma

Studentnummer(s)
496205 (Michyl Stuurop) en 490007 (Maarten Meindertsma)

Vak

Software Defined Networking Practicum

Docent/begeleider
J. Bredek

Datum van inlevering
27/10/2026

Inhoudsopgave

2. Korte inleiding / Introductie 5

3. Theoretisch kader 6

3.1 SDN, OpenFlow en de gebruikte componenten 6

3.1.1 Software Defined Networking 6

3.1.2 OpenFlow 1.3 6

3.1.3 Faucet en de tabelstructuur 7

3.1.4 Mininet en Open vSwitch 7

3.1.5 MAC-learning en flooding 8

3.2 Segmentatie, adressering en routering 8

3.2.1 VLAN's en trunking (IEEE 802.1Q) 8

3.2.2 Routering tussen VLAN's: router-on-a-stick 9

3.2.3 IPv4-adressering en schaalbaarheid 9

3.2.4 NAT en PAT 10

3.2.5 IPv6 10

3.3 NFV en beveiliging 11

3.3.1 NFV en de gekozen functies 11

3.3.2 Rate limiting met meters 11

3.3.3 Port security 12

3.3.4 ACL's en prioriteiten in Faucet 12

3.3.5 Firewalls: stateless en stateful 13

3.3.6 Gelaagde beveiliging 14

3.4 Overlay-netwerken en VxLAN 14

3.4.1 Overlay en underlay 14

3.4.2 VxLAN (RFC 7348) 14

3.4.3 Encapsulatie en MTU 15

3.4.4 Overlays in Proxmox VE SDN 15

4. Eindproduct 16

4.1 Schematische netwerkweergave 16

4.2 Flow entries 18

4.2.1 VLAN-segmentatie en trunking 18

4.2.2 MAC-learning en forwarding 19

4.2.3 Flooding, ARP en IPv6 Neighbor Discovery 19
4.2.4 Gastisolatie 20
Hoe dit bijdraagt. 21
4.2.5 Rate limiting 21
Hoe dit bijdraagt 21
4.2.6 Port security (management) 21
Hoe dit bijdraagt 21
4.2.7 Verkeer tussen verdiepingen, gebouwen en naar de gateway 22
Hoe dit bijdraagt 22
4.2.8 De stateful firewall en NAT zijn geen flow entries 22
4.2.9 Het pad van een pakket: gast naar ISP 23
4.2.10 Samenvatting 24
4.3 Onderbouwing ontwerpkeuzes 25
4.3.1 Topologie: hiërarchisch met een core en een access-laag per gebouw 25
4.3.2 Eén Faucet-controller voor beide gebouwen 26
4.3.3 Faucet als controller 26
4.3.4 Segmentatie en adressering 27
4.3.5 Routering op een Linux-gateway (natgw) in plaats van in Faucet 27
4.3.6 Stateful firewall met conntrack op natgw 28
4.3.7 NAT/PAT op één publiek adres 28
4.3.8 Rate limiting met een OpenFlow-meter 29
4.3.9 Port security met een ACL op bron-MAC 29
4.3.10 Samenvatting 30
4.3.11 Beperkingen en aanbevelingen voor productie 30
4.4 Functioneel bewijs 31
4.4.1 Flow entries, meter en firewall (figuur 0 tot 15) 31
4.4.2 Connectiviteit en bandbreedte (ping, ping6 en iperf) 36
4.4.3 Route naar buiten (traceroute) 37
4.4.4 Pakketten op natgw (tcpdump) 38
4.5 Opdracht 4: Overlay-netwerk met VxLAN 39
4.5.1 Context en doelstelling 39
4.5.2 Netwerktekening en architectuur 39

4.5.3 Onderbouwing ontwerpkeuzes 39
4.5.4 MTU-berekening 40
4.5.5 Bewijs en tests 40
5 .Reflectie en evaluatie 43
6.Literatuurlijst 45
Bijlage 1 – Logboek 46
Bijlage 2 – Code 48
Bijlage 2.1 – Faucet YAML 48
Bijlage 2.2 – Mininet Python 53
Bijlage 2.3 – Overige code/configuratie 58
Uitvoeren en meten (in de Mininet-CLI): 59
show_flows.sh 59
Bijlage 3 Verantwoording gebruik van AI 64

2. Korte inleiding / Introductie

Het CvB wil af van de verouderde Cisco-infrastructuur en onderzoekt een Software Defined Networking-oplossing voor twee gebouwen: gebouw A met twee verdiepingen en gebouw B met drie. De gebouwen zijn met een darkfiber verbonden en het netwerk moet minimaal 500 gelijktijdige gebruikers aankunnen. Voor de verbinding naar buiten is één publiek /28-blok beschikbaar.

Het doel van deze opdracht is een werkend prototype te bouwen en de gemaakte keuzes te onderbouwen. Ik heb het netwerk nagebouwd in Mininet met Open vSwitch. Een Faucet-controller beheert alle zeven switches via OpenFlow 1.3. Het netwerk is opgedeeld in drie VLAN's: werknemers, gasten en management. Daarbovenop heb ik twee NFV-functies gebouwd: rate limiting voor gasten en port security op het managementnetwerk. Het netwerk werkt met IPv4 en IPv6. De routering, NAT en een stateful firewall draaien op een aparte Linux-gateway.

Dit portfolio beschrijft eerst de theorie achter de gekozen oplossing (hoofdstuk 3). Daarna volgt het eindproduct (hoofdstuk 4): de netwerktekening, de flow entries, de onderbouwing van de ontwerpkeuzes en het functionele bewijs met tests en dumps. Het sluit af met een reflectie op het proces (hoofdstuk 5). Opdracht 4 (het overlay-netwerk met VxLAN) heb ik samen met Maarten Meindertsma uitgevoerd; die uitwerking staat in paragraaf 4.5.

3. Theoretisch kader

Dit hoofdstuk beschrijft de theorie achter de gekozen oplossing.

3.1 SDN, OpenFlow en de gebruikte componenten

3.1.1 Software Defined Networking

In een traditioneel netwerk, zoals de oude Cisco-infrastructuur van het CvB, heeft elke switch zowel een control plane als een data plane. De control plane bepaalt hoe verkeer moet lopen, met MAC-learning, spanning tree en routeringsprotocollen. De data plane voert dat uit door pakketten door te sturen. Elk apparaat neemt zijn eigen beslissingen en wordt apart geconfigureerd.

SDN scheidt deze twee lagen. De switches houden alleen de data plane. De besluitvorming verhuist naar een centrale controller, die de switches programmeert via een open protocol (southbound interface). Dit geeft drie voordelen:

- een centraal overzicht en één plek voor beleid;
- configuratie als code, dus reproduceerbaar en te versiebeheren;
- minder afhankelijkheid van één leverancier, omdat switches een standaardprotocol spreken.

Koppeling met de opdracht. Het CvB wil af van de verouderde Cisco-omgeving. In de oplossing is de Faucet-controller op een Vm de control plane. De zeven Open vSwitch-instanties in Mininet vormen de data plane. De volledige netwerkconfiguratie staat in één YAML-bestand, dat op GitHub staat.

3.1.2 OpenFlow 1.3

OpenFlow is het protocol waarmee de controller de flow tables van een switch beheert. Een flow table bestaat uit flow entries met vier onderdelen:

Onderdeel	Betekenis
Match	aan welke velden het pakket moet voldoen (poort, MAC, VLAN, IP, protocol)
Priority	bij meerdere treffers wint de regel met de hoogste prioriteit
Instructions	wat er gebeurt: acties uitvoeren (output, drop, push_vlan, set_field), naar een volgende tabel gaan (goto_table) of een meter toepassen (meter)
Counters	aantal pakketten en bytes dat de regel heeft getroffen

OpenFlow 1.3 ondersteunt meerdere tabellen in een pipeline. Een pakket doorloopt de tabellen op volgorde en elke tabel kan het doorgeven met goto_table. Een pakket dat in een tabel geen regel treft, valt op de table-miss entry (prioriteit 0), die het pakket meestal dropt.

3.1.3 Faucet en de tabelstructuur

Faucet is een OpenFlow-controller die geen losse flows programmeert, maar een YAML-configuratie vertaalt naar een vaste pipeline. De beheerder beschrijft VLAN's, poorten en ACL's, en Faucet bepaalt de flow entries. De configuratie is declaratief: ik beschrijf de gewenste situatie en niet de stappen.

Faucet gebruikt alleen de tabellen die nodig zijn. Op mijn switches zijn dat:

Tabel (verdiepingsswitch)	Functie
0	poort-ACL (filtering en meters)
1	VLAN-classificatie
2	bron-MAC leren (eth_src)
3	doorsturen op doel-MAC (eth_dst)
4	flooding van broadcast en onbekende bestemmingen

Faucet identificeert een switch aan de hand van het datapath-ID (DPID). Een switch met een onbekend DPID wordt geweigerd.

3.1.4 Mininet en Open vSwitch

Mininet emuleert een netwerk op één machine. Hosts zijn Linux network namespaces, verbindingen zijn virtuele ethernetparen (veth) en switches zijn Open vSwitch-instanties. Dit geeft realistisch gedrag van de Linux-netwerkstack (inclusief iptables en conntrack), maar geen realistische doorvoer. Alle verkeer loopt via de CPU van de VM.

Koppeling met de opdracht.

Dit heeft drie gevolgen voor mijn werk:

1. De iperf-waarden van tientallen Gbit/s zijn softwaresnelheden. Ik gebruik ze alleen vergelijkend (gast tegenover werknemer), niet als prestatiemeting.
2. Linux beperkt interfacenamen tot 15 tekens. Mijn eerste hostnamen (a_f1_employee-eth0) waren te lang, waardoor Mininet faalde. Daarom gebruik ik korte namen als a1_emp.
3. Omdat natgw een volwaardige Linux-namespace is, kan ik daar echte iptables-regels en conntrack gebruiken.

3.1.5 MAC-learning en flooding

Een switch leert bron-MAC-adressen en onthoudt op welke poort ze zitten. Bekende doelen worden direct doorgestuurd, onbekende en broadcastverkeer worden geflood naar alle poorten van het VLAN.

Koppeling met de opdracht.

Faucet doet dit in tabel 2 (eth_src) en 3 (eth_dst). Een onbekend bron-MAC wordt met CONTROLLER:96 naar de controller gestuurd, die er een leerregel voor installeert (zichtbaar met hard_timeout en idle_timeout). Tabel 4 bevat de floodregels per VLAN.

3.2 Segmentatie, adressering en routering

3.2.1 VLAN's en trunking (IEEE 802.1Q)

Een VLAN is een logisch gescheiden layer 2-domein op gedeelde hardware. Een tag van 4 bytes in het ethernetframe draagt het VLAN-ID (12 bits, dus 4094 bruikbare waarden). Er zijn twee soorten poorten:

- Access-poort: de host stuurt ongetagd verkeer, de switch wijst het toe aan één VLAN.
- Trunkpoort: draagt meerdere VLAN's tegelijk, met tags.

Een broadcast blijft binnen het eigen VLAN. Zo ontstaat segmentatie op layer 2: hosts in verschillende VLAN's kunnen elkaar niet rechtstreeks bereiken, ook al hangen ze aan dezelfde switch.

Koppeling met de opdracht.

In Faucet is dit de configuratie:

Faucet-optie	**Rol**	**Waar**
native_vlan: guest	access-poort	poorten 2 t/m 4 op de verdiepingsswitches
tagged_vlans: [office, guest, management]	trunk	poort 1 op de verdiepingen, de coreverbindingen en de darkfiber

Drie VLAN's scheiden de doelgroepen: 100 (werknemers), 200 (gasten) en 300 (management). De darkfiber tussen de gebouwen is een trunk. Daardoor bevinden een werknemer in gebouw A en een werknemer in gebouw B zich in hetzelfde layer 2-domein.

In de flows is het VLAN zichtbaar als set_field:4196->vlan_vid. OpenFlow telt het bit OFPVID_PRESENT (0x1000 = 4096) bij het VLAN-ID op, dus 4096 + 100 = 4196 voor VLAN 100, 4296 voor VLAN 200 en 4396 voor VLAN 300.

3.2.2 Routering tussen VLAN's: router-on-a-stick

VLAN's zijn gescheiden domeinen. Verkeer tussen VLAN's moet via een router lopen, waar beleid kan worden afgedwongen. Bij router-on-a-stick staat de router aan één trunk-verbinding en heeft hij per VLAN een subinterface (eth0.100, eth0.200) met het gateway-adres van dat VLAN.

Koppeling met de opdracht.

Faucet kan zelf routeren (via faucet_vips). Dit is bewust niet voor gekozen. Want Faucet routeert per datapath. In een netwerk met zeven switches had dat veel complexiteit gegeven, terwijl routering en beleid dan over meerdere plekken verspreid raken.

In plaats daarvan is natgw router en firewall. Het hangt aan a_core poort 4, die een trunk met alle drie de VLAN's is. Daarop staan de subinterfaces natgw-eth0.100, .200 en .300 met de gateways 10.0.100.254, 10.0.200.254 en 10.0.30.254. Alle inter-VLAN-verkeer en al het verkeer naar buiten loopt dus langs één punt. Dat maakt het beleid overzichtelijk en aantoonbaar met tellers.

3.2.3 IPv4-adressering en schaalbaarheid

Een IPv4-subnet met prefix /n heeft 2^(32−n) − 2 bruikbare hostadressen. Een /24 heeft er 254, een /23 heeft er 510 en een /28 heeft er 14..

Koppeling met de opdracht.

De eis van minimaal 500 gelijktijdige gebruikers bepaalt de adressering:

Netwerk	Subnet	Bruikbare adressen	Reden
Employee	10.0.100.0/23	510	past bij de eis van 500 gebruikers
Guest	10.0.200.0/24	254	gastaantallen zijn kleiner
Management	10.0.30.0/24	254	beperkt aantal beheerhosts
Publiek	198.51.100.0/28	14	het enige publieke blok dat beschikbaar is

Een /24 voor werknemers (254 adressen) was te klein geweest. Een /22 zou meer adressen opleveren dan nodig is. Een grote broadcast-domein kost bovendien capaciteit, dus de /23 is de kleinste passende keuze.

De adressen in het publieke segment (198.51.100.0/24, RFC 5737) en voor IPv6 (2001:db8::/32, RFC 3849) zijn gereserveerd voor documentatie en testen. Dat past bij een prototype. In productie zijn dit de adressen van de echte ISP.

3.2.4 NAT en PAT

Omdat het publieke blok klein is (14 bruikbare adressen), kunnen niet alle interne hosts een eigen publiek adres krijgen. Bij PAT vervangt de gateway het bronadres van uitgaand verkeer door één publiek adres en wijst hij elke verbinding een eigen bronpoort toe. De gateway onthoudt de vertaling in een conntrack-tabel en past die toe op het terugverkeer. Een poort is 16 bits, dus één publiek adres kan in de orde van tienduizenden gelijktijdige verbindingen per bestemming dragen. Een bijwerking is dat van buitenaf niemand een nieuwe verbinding naar een intern adres kan beginnen, omdat er geen vertaalregel voor bestaat.

Koppeling met de opdracht.

Hiermee los ik de /28-beperking op. De regel op natgw is:

iptables -t nat -A POSTROUTING -s 10.0.0.0/8 -o natgw-eth1 -j SNAT --to-source 198.51.100.2

Alle interne IPv4-hosts delen daardoor 198.51.100.2.

In de 500-gebruikerstest stegen de SNAT-teller van 3 naar 503 en de conntrack-tabel naar 500 vermeldingen, terwijl 500 van 500 bronadressen 0% verlies hadden. De beperking is dat de bronadressen van één host komen, met één MAC. De test bewijst PAT en conntrack, niet het leren van 500 MAC's in Faucet.

3.2.5 IPv6

IPv6 heeft een 128-bits adres. Een standaard-subnet is een /64, en er is zoveel adresruimte dat NAT niet nodig is: elke host kan een globaal routeerbaar adres hebben (RFC 8200). IPv6 vervangt ARP door Neighbor Discovery (RFC 4861), dat via ICMPv6 loopt. De belangrijkste berichten zijn Neighbor Solicitation (type 135) en Neighbor Advertisement (type 136). Deze worden naar multicastadressen gestuurd die beginnen met 33:33: op layer 2.

Omdat er geen NAT is, wordt het verkeer alleen beschermd door de firewall. Dat zou een gat zijn als nieuwe inkomende verbindingen niet expliciet zouden worden geblokkeerd.

Koppeling met de opdracht.

Elk VLAN heeft een eigen /64 met natgw als gateway op ::1. Voor IPv6 pas ik geen NAT toe: de prefixen worden gerouteerd en de ISP-hosts hebben terugroutes via 2001:db8:ffff::2. De firewall zorgt dat nieuwe verbindingen van buiten niet binnenkomen.

Twee praktische gevolgen zie ik terug in de configuratie:

- De gast-ACL moet ICMPv6-types 135 en 136 expliciet toelaten (icmpv6_type: 135 en 136). Zonder deze regels vindt een gast zijn gateway niet en werkt IPv6 niet, ook al is verkeer naar de gateway verder toegestaan.
- In de flow-dumps zie je de multicast-flooding van ND terug (dl_dst=33:33:00:00:00:00/ff:ff:00:00:00:00).

Hosts hebben statische IPv6-adressen. In productie zou je SLAAC of DHCPv6 gebruiken.

3.3 NFV en beveiliging

3.3.1 NFV en de gekozen functies

NFV (Network Functions Virtualization) draait netwerkfuncties, zoals firewalls, NAT en load balancers, als software in plaats van op dedicated hardware. Het doel is flexibiliteit en lagere kosten, en dat sluit aan bij de opdracht van het CvB om kosten te besparen.

Koppeling met de opdracht.

De opdracht noemt twee verplichte NFV-functies en ik voeg er twee toe:

Functie	**Waar uitgevoerd**	**Mechanisme**
Rate limiting gasten	in de datapath (OVS)	OpenFlow-meter
Port security management	in de datapath (OVS)	Faucet-ACL op bron-MAC
NAT/PAT	softwarefunctie op natgw	iptables SNAT
Stateful firewall	softwarefunctie op natgw	iptables, ip6tables en conntrack

Eerlijk geformuleerd zijn NAT en firewall op natgw de zuiverste NFV-voorbeelden: functies als software op een standaard Linux-host. Rate limiting en port security worden in de switch zelf uitgevoerd, als OpenFlow-functie.

3.3.2 Rate limiting met meters

Een OpenFlow-meter werkt als een token bucket. Er komen tokens binnen op de ingestelde snelheid, elk pakket verbruikt tokens en als de emmer leeg is, past de band zijn actie toe (hier: DROP). De parameter burst bepaalt hoeveel pieken de emmer toestaat.

Mijn meter is rate: 1000 (KBPS, dus 1 Mbit/s) met burst_size: 100. De flow koppelt hem met actions=meter:1. Een gast haalt daardoor circa 1,1 tot 1,5 Mbit/s, tegenover 15 Gbit/s voor een werknemer. Het afwijken boven de 1000 kbit/s komt door de burst en doordat iperf TCP-gedrag meet.

Er is kozen voor een meter boven tc omdat de limiet daarmee in de switch zelf zit, door Faucet wordt beheerd en zichtbaar is in dump-meters en meter-stats.

3.3.3 Port security

Bij port security accepteert een poort alleen verkeer van vooraf bekende MAC-adressen. Dat voorkomt dat iemand een niet-geautoriseerd apparaat op een managementpoort aansluit.

Mijn implementatie is een ACL per poort met een allow-regel op eth_src en een drop-regel voor al het overige. In de flows staat de catch-all met een lagere prioriteit dan de allow:

priority=20480,in_port="a_f1-eth4",dl_src=02:00:00:00:30:11 actions=goto_table:1

priority=20479,in_port="a_f1-eth4" actions=drop

De beperking is dat MAC-adressen te vervalsen zijn. Port security is daarom een eerste hindernis en geen volledige authenticatie.

3.3.4 ACL's en prioriteiten in Faucet

Een ACL is een lijst regels die van boven naar beneden wordt beoordeeld, waarbij de eerste treffer geldt (first match). Faucet vertaalt de volgorde naar aflopende OpenFlow-prioriteiten.

Koppeling met de opdracht.

De volgorde in de gast-ACL is daarom ontwerpbeslissend. Mijn dump toont dat:

Prioriteit	**Regel**
20480 tot 20478	ARP en ND toestaan
20477	naar eigen gateway toestaan (meter)
20476 tot 20474	drop naar guest-, employee- en management-prefix (IPv4)
20473	rest naar het gateway-MAC toestaan, dus naar de ISP (meter)
20472 tot 20468	IPv6-equivalenten
20467	alles anders droppen

Het verkeer naar de ISP is een regel die op het doel-MAC van natgw (02:00:00:00:fe:01) matcht. Een gast die probeert rechtstreeks met een andere gast te praten, stuurt pakketten naar een ander MAC en valt op de drop. De drops voor de eigen prefixen staan boven de ISP-regel, zodat ook een gesmeed pakket met het gateway-MAC maar een intern doel niet doorkomt.

3.3.5 Firewalls: stateless en stateful

Een stateless filter beoordeelt elk pakket los. Het kan niet weten of een pakket bij een bestaande verbinding hoort. Een stateful firewall houdt een verbindingstabel bij (connection tracking) en kent vier toestanden:

Status	Betekenis
NEW	eerste pakket van een nieuwe verbinding
ESTABLISHED	pakket hoort bij een bekende verbinding
RELATED	hoort bij een bestaande verbinding (bijvoorbeeld een ICMP-foutmelding)
INVALID	past in geen enkele verbinding

Met de regel “sta alleen NEW van binnen naar buiten toe en ESTABLISHED/RELATED in beide richtingen” werkt het terugverkeer vanzelf, maar kan niemand van buiten een verbinding starten.

Waarom niet in OpenFlow? OpenFlow 1.3 kent geen connection tracking. In Faucet is die niet beschikbaar. Een stateless nabootsing, bijvoorbeeld “sta TCP-pakketten met ACK-vlag toe”, werkt niet voor UDP en ICMP en is te omzeilen met gesmeed verkeer. Daarom draait de stateful logica op natgw, waar Linux netfilter met conntrack echte verbindingsstatus bijhoudt voor IPv4 en IPv6.

Koppeling met de opdracht.

De regels op natgw zijn (volgorde van boven naar beneden, standaardbeleid DROP):

1. ctstate INVALID: drop.
2. ctstate ESTABLISHED,RELATED: accept.
3. ctstate NEW van VLAN 100, 200 en 300 naar de ISP-kant: accept.
4. ctstate NEW van management naar employee: accept.

Een voorbeeld van waarom dit nodig is, kwam uit eigen werk. De eerste versie blokkeerde employee naar management met een stateless Faucet-ACL op de werknemerspoort. Daardoor werd ook het antwoord van een werknemer op een management-ping gedropt, en kon management zijn eigen verbindingen niet voltooien. Met de stateful firewall hoeft alleen de richting van de nieuwe verbinding te worden toegestaan. Het terugverkeer volgt vanzelf. Daarom zit het beleid tussen interne VLAN’s nu op natgw en niet in de switch-ACL’s.

3.3.6 Gelaagde beveiliging

Beveiliging op meerdere niveaus voorkomt dat één fout het hele beleid ongedaan maakt.

Koppeling met de opdracht.

Voor gastisolatie heb ik twee lagen:

- laag 1: de gast-ACL op de poort, die gast-naar-gast-verkeer in hetzelfde VLAN blokkeert. Dat verkeer bereikt natgw nooit, dus alleen de switch kan het stoppen;
- laag 2: de firewall op natgw, die gastverkeer naar andere VLAN's weigert.

De twee lagen vullen elkaar aan en de tellers van beide zijn zichtbaar in het bewijsmateriaal.

3.4 Overlay-netwerken en VxLAN

3.4.1 Overlay en underlay

Een overlay-netwerk is een virtueel netwerk dat bovenop een bestaand netwerk, de underlay, wordt gebouwd. De underlay zorgt alleen voor IP-connectiviteit (Laag 3) tussen de eindpunten. De overlay legt daar een eigen Laag 2-netwerk overheen. Hosts in de overlay zien elkaar alsof ze aan dezelfde switch hangen, ook als er tussen de locaties meerdere routers liggen.

Dit verschilt van een VLAN. Een VLAN (IEEE 802.1Q, paragraaf 3.2.1) werkt binnen één Laag 2-domein en gebruikt een 12-bits ID, dus maximaal 4094 netwerken. Een overlay werkt over Laag 3 heen en kan veel meer netwerken aan. Het is ook een andere vorm van SDN dan OpenFlow: er is geen controller die flow entries in switches zet. De virtuele netwerken worden centraal gedefinieerd en daarna door de hosts zelf opgebouwd.

Koppeling met de opdracht.

In opdracht 4 zijn de underlay de twee gerouteerde netwerken van de Proxmox-clusters (10.24.11.0/24 en 10.24.20.0/24). De overlay is het subnet 172.16.100.0/24, waarin een VM van het ene cluster en een VM van het andere cluster direct met elkaar communiceren.

3.4.2 VxLAN (RFC 7348)

VxLAN (Virtual eXtensible LAN) is een overlayprotocol dat Ethernet-frames inpakt in UDP-pakketten (MAC-in-UDP). Het eindpunt dat inpakt en uitpakt heet een VTEP (VxLAN Tunnel Endpoint). De belangrijkste kenmerken zijn:

• Elk virtueel netwerk heeft een VNI (VxLAN Network Identifier) van 24 bits. Dat zijn ruim 16 miljoen netwerken, tegenover 4094 bij VLAN's.

• Het verkeer reist als UDP met bestemmingspoort 4789. Die poort moet dus open staan tussen de VTEP's.

• Een VTEP leert, net als een switch, welk MAC-adres achter welke andere VTEP zit. Broadcast, onbekend unicast en multicast (BUM-verkeer) wordt gerepliceerd naar de andere VTEP's, bijvoorbeeld via multicast of via een lijst met peers.

• Omdat de underlay alleen UDP/IP ziet, kan de overlay over bestaande routers lopen zonder dat die iets van VxLAN hoeven te weten.

Koppeling met de opdracht.

De Proxmox-nodes zijn zelf de VTEP's en kennen elkaar via een peers-lijst met de vmbr0-adressen van beide clusters. Het netwerk gebruikt VNI 10100. Dat de underlay UDP 4789 doorlaat, bleek uit onze tests en was de reden om geen extra VPN-laag te gebruiken.

3.4.3 Encapsulatie en MTU

Het inpakken kost extra bytes. Voor elk frame komen er een buiten-IPv4-header (20 bytes), een UDP-header (8 bytes) en een VxLAN-header (8 bytes) bij, en het binnen-Ethernetframe heeft zelf een header van 14 bytes. In totaal is dat 50 bytes overhead. Bij een underlay-MTU van 1500 bytes blijft er voor de overlay 1500 − 50 = 1450 bytes over. Is de MTU in de overlay groter, dan moet de underlay het pakket fragmenteren, of het pakket gaat verloren als het DF-bit (Don't Fragment) staat.

Koppeling met de opdracht.

We hebben de MTU van de zone en de VM-netwerkkaarten op 1450 gezet. Met een ping met het DF-bit en een payload van 1422 bytes (1450 min 28 bytes IP- en ICMP-header) tonen we aan dat dit klopt (test M3).

3.4.4 Overlays in Proxmox VE SDN

Proxmox VE heeft een eigen SDN-functie voor overlays, beschreven in de Proxmox VE Administration Guide. Die kent drie onderdelen:

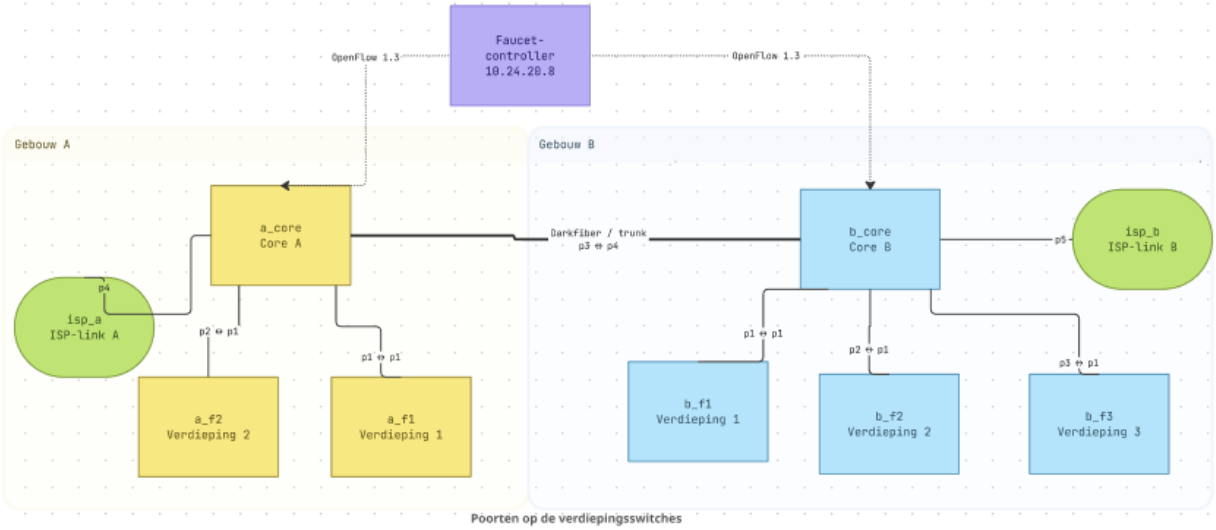
• Zone: het type netwerk en de instellingen die voor alle netwerken erin gelden, zoals het type (VxLAN), de peers-lijst en de MTU.

• VNet: een virtueel netwerk binnen een zone. Het is op de node een Linux-bridge waar VM's aan worden gekoppeld. Bij een VxLAN-zone is de tag van het VNet de VNI.

• Subnet: het IP-bereik en de gateway van een VNet, eventueel met IPAM.

4. Eindproduct

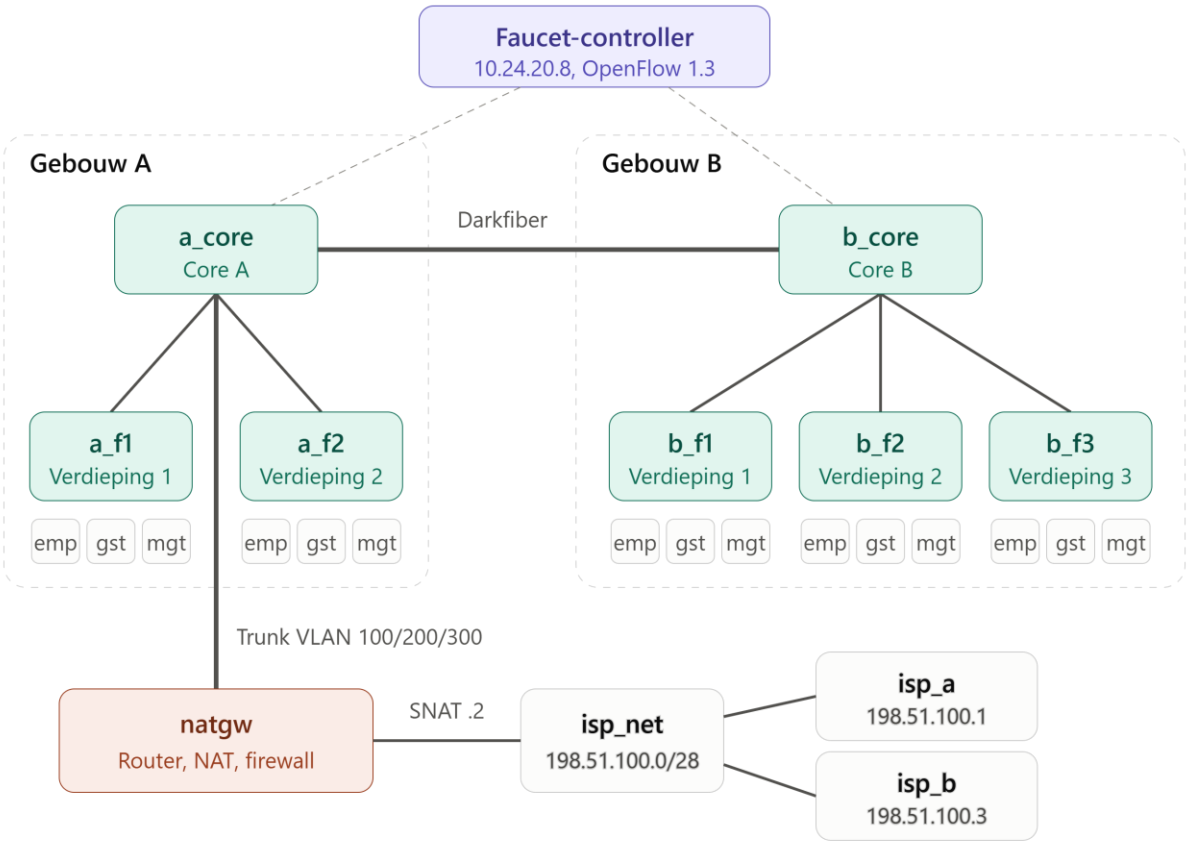
4.1 Schematische netwerkweergave



Poorten op de verdiepingsswitches
Poort 1 = trunk naar de core-switch
Poort 2 = employee/office-host
Poort 3 = guest-host
Poort 4 = management-host

VLANs
VLAN 100 = office / employees
VLAN 200 = guest
VLAN 300 = management

Switches
a_core / b_core = centrale switch
a_f1, b_f1 = verdieping 1
a_f2, b_f2 = verdieping 2
b_f3 = verdieping 3



Poort 1 trunk, poort 2 emp (VLAN 100), poort 3 gst (VLAN 200), poort 4 mgt (VLAN 300)
gst: meter 1 Mbit/s plus ACL naar gateway | mgt: port security, vast MAC per poort
natgw: stateful firewall (conntrack), SNAT naar 198.51.100.2, IPv6 gerouteerd zonder NAT

4.2 Flow entries

Elke functionaliteit in mijn netwerk steunt op flow entries die Faucet in de zeven switches heeft geïnstalleerd. Ze komen op twee manieren tot stand:

- Statisch, uit de YAML. Faucet maakt ze direct bij het verbinden van de switch: de poort-ACL's (tabel 0), de VLAN-classificatie (tabel 1) en de floodregels (laatste tabel).
- Dynamisch, door leren. Faucet maakt ze nadat een pakket met een onbekend bron-MAC naar de controller is gestuurd (CONTROLLER:96): de bron-MAC-regels en de doorstuurregels. Ze hebben een time-out (hard_timeout en idle_timeout) en verdwijnen als het MAC niet meer gebruikt wordt.

Alle voorbeelden komen uit mijn dumps van a_f1 en a_core (zie bewijsfiguren 1 tot 10). De tellers (n_packets) laten zien dat de regels daadwerkelijk zijn geraakt.

4.2.1 VLAN-segmentatie en trunking

Flow entry (ingekort)	Tabel	Teller
in_port="a_f1-eth2", vlan_tci=0x0000/0x1fff → push_vlan:0x8100, set_field:4196->vlan_vid, goto_table:2	1	186978
in_port="a_f1-eth3", vlan_tci=0x0000/0x1fff → push_vlan, set_field:4296->vlan_vid, goto_table:2	1	1004
in_port="a_f1-eth4", vlan_tci=0x0000/0x1fff → push_vlan, set_field:4396->vlan_vid, goto_table:2	1	71
in_port="a_f1-eth1", dl_vlan=100 (en 200, 300) → goto_table:2	1	132400 / 787 / 110
priority=0 → drop	1	0

Hoe dit bijdraagt aan het netwerkgedrag.

- Een host stuurt ongetagd verkeer. De eerste drie regels (vlan_tci=0x0000/0x1fff, dus "geen tag") voegen een tag toe op basis van de poort. De poort bepaalt het VLAN, niet de host. Een gastpoort kan dus nooit verkeer in VLAN 100 plaatsen.
- Op de trunkpoort (eth1) accepteren de regels alleen VLAN 100, 200 en 300. Een frame met een andere tag valt op de priority=0-regel en wordt gedropt. Dat is het "default deny"-gedrag.
- De waarde 4296 is 0x1000 plus 200. Het bit 0x1000 betekent "VLAN-tag aanwezig". Vanaf tabel 2 matcht alles op dl_vlan, dus het hele netwerk werkt per VLAN.

4.2.2 MAC-learning en forwarding

Flow entry (ingekort)	Tabel	Teller
in_port="a_f1-eth3", dl_vlan=200, dl_src=00:00:00:00:00:02 → goto_table:3	2	13
dl_vlan=200 → CONTROLLER:96, goto_table:3	2	6
dl_vlan=200, dl_dst=00:00:00:00:00:02 → pop_vlan, output:"a_f1-eth3"	3	788
dl_vlan=200, dl_dst=02:00:00:00:fe:01 → output:"a_f1-eth1"	3	1007
priority=0 → goto_table:4	3	143

Hoe dit bijdraagt.

- De eerste regel is de uitkomst van het leren: Faucet heeft gezien dat a1_gst (MAC ...:02) op poort 3 in VLAN 200 zit. Zo'n regel vervalt na een hard_timeout.
- Een nieuw, onbekend MAC valt op de CONTROLLER:96-regel (priority 4096). De controller ontvangt het pakket en installeert de leerregel. Daarom ziet de tweede regel maar 6 pakketten, terwijl de geleerde regel er 13 heeft.
- In tabel 3 is het verschil zichtbaar tussen een hostpoort en een trunk. Naar a1_gst (poort 3) gaat pop_vlan, dus de tag gaat eraf. Naar natgw via de trunk (eth1) blijft de tag erop, zodat natgw het VLAN kan herkennen.
- Wat niet gevonden wordt in tabel 3 (priority=0) gaat naar tabel 4, de flooding.

4.2.3 Flooding, ARP en IPv6 Neighbor Discovery

Flow entry (ingekort)	Tabel	Teller
dl_vlan=200, dl_dst=ff:ff:ff:ff:ff:ff → output:"a_f1-eth1", pop_vlan, output:"a_f1-eth3"	4	2
dl_vlan=200, dl_dst=33:33:00:00:00:00/ff:ff:00:00:00:00 → output:eth1, pop_vlan, output:eth3	4	9
dl_dst=01:00:0c:cc:cc:cc en 01:80:c2:00:00:00/... → drop	4	0
priority=0 → drop	4	0

Hoe dit bijdraagt.

- Een broadcast (ARP) in VLAN 200 gaat alleen naar de trunk en de gastpoort van dat VLAN, nooit naar de poorten van VLAN 100 of 300. Dit is de layer 2-scheiding.
- IPv6 gebruikt geen broadcast maar multicast met een MAC dat begint met 33:33. De aparte floodregel daarvoor (9 pakketten voor VLAN 200) zorgt dat Neighbor Discovery werkt.
- De drops voor 01:00:0c:cc:cc:cc (Cisco-protocollen) en 01:80:c2:... (spanning tree en LLDP) voorkomen dat beheerverkeer van andere apparatuur door het netwerk lekt.
- In tabel 2 staan ook beveiligingsregels van Faucet zelf, zoals dl_src=ff:ff:ff:ff:ff:ff → drop (een broadcast kan nooit een bron zijn), dl_src=0e:00:00:00:00:01 → drop (het eigen MAC van Faucet kan niet worden gespoofd) en dl_type=0x9000 → drop (loopback-frames).

4.2.4 Gastisolatie

Flow entry (ingekort, in_port="a_f1-eth3")	**Prioriteit**	**Teller**
arp → goto_table:1	20480	16
icmp6, icmp_type=135 en 136 → goto_table:1	20479 en 20478	4 en 4
ip, dl_dst=02:00:00:00:fe:01, nw_dst=10.0.200.254 → meter:1, goto_table:1	20477	0
ip, nw_dst=10.0.200.0/24 → drop	20476	3
ip, nw_dst=10.0.100.0/23 → drop	20475	3
ip, nw_dst=10.0.30.0/24 → drop	20474	3
ip, dl_dst=02:00:00:00:fe:01 → meter:1, goto_table:1	20473	979
ipv6, ipv6_dst=2001:db8:200::/64 → drop (en ...100::/64, ...300::/64)	20471 tot 20469	3 / 0 / 0
ipv6, dl_dst=02:00:00:00:fe:01 → meter:1, goto_table:1	20468	3
in_port="a_f1-eth3" → drop	20467	11

Hoe dit bijdraagt. De regels werken als een lijst die van boven naar beneden wordt afgewerkt (hoogste prioriteit eerst):

1. ARP en Neighbor Discovery mogen altijd door, anders vindt een gast zijn gateway niet.
2. Verkeer naar een ander gastadres, het employee-netwerk of het management-netwerk wordt gedropt (regels 20476 tot 20474). Daarom faalt a1_gst ping 10.0.200.12: de drie gedropte pakketten staan in de teller.
3. Alles wat het MAC-adres van natgw als doel heeft, mag door, met de meter erbij (20473). Dat is het pad naar de ISP. De 979 pakketten zijn het iperf- en pingverkeer.
4. Al het overige valt op de laatste regel (20467) en wordt gedropt.

De drops staan met opzet boven de toelating: een pakket met het gateway-MAC maar een intern doeladres wordt dan nog steeds geblokkeerd. Gast-naar-gast-verkeer binnen hetzelfde VLAN passeert natgw nooit. Alleen deze flow entries op de poort kunnen het stoppen.

4.2.5 Rate limiting

Onderdeel	Inhoud
Meter	meter=1 kbps burst stats bands=type=drop rate=1000 burst_size=100
Flow entries die hem gebruiken	de vier gastregels met actions=meter:1,goto_table:1 (prioriteit 20477, 20473, 20472 en 20468)

Hoe dit bijdraagt. Elk pakket dat een van die regels raakt, gaat eerst door de meter. De meter werkt als een emmer met tokens. Komt het verkeer boven 1000 kbit/s uit, dan is de emmer leeg en past de band zijn actie toe: drop. De overige pakketten gaan door naar tabel 1. Hierdoor haalt de gast 1,08 tot 1,51 Mbit/s, terwijl een werknemer (die niet door de meter gaat) 14,6 Gbit/s haalt. De limiet zit in de datapath van de switch, dus er is geen extra software op de hosts nodig.

4.2.6 Port security (management)

Flow entry	Prioriteit	Teller
in_port="a_f1-eth4", dl_src=02:00:00:00:30:11 → goto_table:1	20480	79
in_port="a_f1-eth4" → drop	20479	3

Hoe dit bijdraagt. De eerste regel laat alleen het geregistreerde MAC-adres door. Alles wat daaronder valt, wordt door de tweede regel gedropt. Toen ik het MAC van a1_mgt veranderde naar 02:00:00:00:99:99, viel elk pakket op de dropregel, en de teller op de dropregel steeg (in figuur 5

staan 3 pakketten). Het pakket komt daardoor nooit in tabel 1, dus het MAC wordt niet geleerd en de host is niet bereikbaar.

4.2.7 Verkeer tussen verdiepingen, gebouwen en naar de gateway

Op a_core (geen ACL's, dus de tabellen zijn 0 = VLAN, 1 = bron-MAC, 2 = doel-MAC, 3 = flooding):

Flow entry (ingekort)	Tabel	Teller	Betekenis
in_port=4, dl_vlan=100 (en 200, 300) → goto_table:1	0	132361 / 795 / 78	verkeer van natgw (trunk)
in_port=3, dl_vlan=100 (en 200, 300) → goto_table:1	0	41 / 0 / 33	verkeer van gebouw B over de darkfiber
dl_vlan=100, dl_dst=02:00:00:00:fe:01 → output:"a_core-eth4"	2	186975	richting natgw
dl_vlan=200, dl_dst=02:00:00:00:fe:01 → output:"a_core-eth4"	2	865	idem, gastverkeer
dl_vlan=200, dl_dst=00:00:00:00:00:05 → output:"a_core-eth2"	2	3	richting de andere verdieping (a_f2)

Hoe dit bijdraagt

Op de core blijven de tags erop (output zonder pop_vlan), omdat alle uitgangen trunks zijn. Het MAC-adres van natgw (02:00:00:00:fe:01) is op a_core per VLAN geleerd op poort 4. Daardoor weet de core dat al het verkeer dat naar de gateway moet, via poort 4 loopt. Verkeer van hosts in gebouw B komt binnen op poort 3, de darkfiber (de in_port=3-regels in tabel 0). Dit zijn dezelfde regels voor alle drie de VLAN's, wat de trunking aantoont.

4.2.8 De stateful firewall en NAT zijn geen flow entries

De stateful firewall en NAT draaien als iptables- en ip6tables-regels op natgw, niet in OpenFlow. Open vSwitch kent wel conntrack-extensies, maar Faucet programmeert die niet. De flow entries zorgen er wel voor dat het verkeer natgw bereikt.

4.2.9 Het pad van een pakket: gast naar ISP

Dit voorbeeld laat zien hoe de regels samen het netwerkgedrag vormen. Het pakket is een ping van a1_gst naar 198.51.100.1:

Stap	Waar	Flow entry	Wat er gebeurt
1	a_f1 tabel 0	ip, in_port=3, dl_dst=...fe:01 → meter:1, goto_table:1	toegestaan, door de meter
2	a_f1 tabel 1	in_port=3, vlan_tci=0x0000/0x1fff → push_vlan, set_field:4296	krijgt VLAN 200
3	a_f1 tabel 2	in_port=3, dl_vlan=200, dl_src=...:02 → goto_table:3	bron-MAC is bekend
4	a_f1 tabel 3	dl_vlan=200, dl_dst=...fe:01 → output:"a_f1-eth1"	uit via de trunk, met tag
5	a_core tabel 0	in_port=1, dl_vlan=200 → goto_table:1	komt binnen op de core
6	a_core tabel 2	dl_vlan=200, dl_dst=...fe:01 → output:"a_core-eth4"	naar natgw
7	natgw	firewall (NEW van VLAN 200 naar buiten) en SNAT	bronadres wordt 198.51.100.2

Het antwoord loopt de omgekeerde weg. Op a_core leidt dl_dst=...:02 naar poort 1, en op a_f1 verwijdert pop_vlan de tag voordat het pakket via poort 3 bij a1_gst aankomt.

Voor een gast die een andere gast pingt (a1_gst naar 10.0.200.12) stopt het pakket al bij stap 1: het doel-MAC is dat van de andere gast, niet van natgw, en het doeladres valt onder nw_dst=10.0.200.0/24 (prioriteit 20476). Het pakket wordt gedropt, nog vóór het geleerd of doorgestuurd kan worden.

4.2.10 Samenvatting

Functionaliteit	Belangrijkste flow entries	Tabel	Bewijs
VLAN-segmentatie	push_vlan en set_field per poort, dl_vlan op trunks	1	figuur 1
MAC-learning en forwarding	geleerde dl_src en dl_dst met pop_vlan of output	2 en 3	figuur 6 en 7
Flooding en IPv6 ND	broadcast- en 33:33-multicastregels per VLAN	4	figuur 8
Gastisolatie	drops op nw_dst en ipv6_dst, toelating op gateway-MAC	0	figuur 2
Rate limiting	meter:1 in de gastregels	0	figuur 3 en 4
Port security	allow op dl_src, catch-all drop	0	figuur 5
Trunk en darkfiber	dl_vlan 100, 200 en 300 op core-poorten	0 (core)	figuur 9 en 10
Stateful firewall en NAT	geen flow entries, wel doorsturen naar het gateway-MAC	n.v.t.	figuur 11 tot 13

Samengevat: tabel 0 beslist wat een poort mag sturen, tabel 1 bepaalt in welk VLAN het hoort, tabel 2 en 3 leren en sturen door, en tabel 4 vangt broadcast en multicast op. Alles wat niet expliciet is toegestaan, valt op een priority=0 drop.

4.3 Onderbouwing ontwerpkeuzes

Elke keuze is afgewogen op de vier criteria uit de opdracht: prestaties, schaalbaarheid, complexiteit en beheer.

4.3.1 Topologie: hiërarchisch met een core en een access-laag per gebouw

Keuze. Elk gebouw heeft één core-switch en een access-switch per verdieping. De cores zijn onderling verbonden via de darkfiber. Dit volgt de beschrijving van de patchkasten: elke verdieping heeft een eigen switch, die uitkomt op een centrale patchkast.

Alternatieven.

- Eén grote switch per gebouw: past niet bij de fysieke situatie, want elke verdieping heeft een eigen patchkast.
- Redundante ring of mesh tussen de verdiepingen: robuuster, maar vraagt loopbeveiliging en meer poorten.

Criterium	**Afweging**
Prestaties	Verkeer op dezelfde verdieping en binnen hetzelfde VLAN blijft layer 2. Het hoeft niet langs een router (ttl 64 bij a1_emp naar b3_emp)
Schaalbaarheid	Een verdieping erbij is één switch en één trunkpoort op de core
Complexiteit	Een boomstructuur heeft geen lussen, dus geen spanning tree nodig
Beheer	Elke switch heeft dezelfde poortindeling (1 trunk, 2 employee, 3 gast, 4 management), wat de configuratie voorspelbaar maakt

Nadeel. Een uitgevallen core of uplink legt een gebouw of verdieping plat. In productie is een tweede uplink per verdieping de aangewezen uitbreiding.

4.3.2 Eén Faucet-controller voor beide gebouwen

Keuze. Eén controller beheert alle zeven switches.

Alternatief. Een controller per gebouw (de opdracht noemt dit als optie), of een redundante clusteropzet.

Criterium	Afweging
Prestaties	De controller zit alleen in het leerpad (nieuwe MAC-adressen). Verkeer van bestaande flows loopt in de switch zelf
Schaalbaarheid	Eén controller is voor dit aantal switches ruim voldoende. Bij groei is clustering of een controller per gebouw mogelijk
Complexiteit	Eén YAML-bestand en een logbestand
Beheer	Eén plek voor beleid. Twee controllers zouden dezelfde VLAN- en ACL-definities moeten delen en kunnen uit de pas lopen

Nadeel. De controller is een single point of failure voor nieuwe flows. Bestaande flows blijven werken tot hun timeout, maar nieuwe hosts worden niet meer geleerd.

4.3.3 Faucet als controller

Keuze. Faucet (1.10.12).

Criterium	Afweging
Prestaties	Faucet installeert een vaste pipeline van flows. Er is geen applicatielogica per pakket
Schaalbaarheid	Gebouwd voor productienetwerken met veel switches
Complexiteit	Declaratieve YAML in plaats van programmeren.
Beheer	Configuratie als code

Nadeel. Faucet is minder flexibel. Het kent geen connection tracking. Daarom staat de stateful firewall op een apart onderdeel (paragraaf 4.3.6).

4.3.4 Segmentatie en adressering

Keuze. Drie VLAN's (100 werknemers, 200 gasten, 300 management) met een /23, /24 en /24, plus een eigen IPv6 /64 per VLAN.

Criterium	Afweging
Prestaties	Een /23 maakt een groter broadcast-domein (tot 510 hosts). Dat is acceptabel voor 500 gebruikers, maar groter dan dit schaalt minder goed
Schaalbaarheid	De /23 is de kleinste passende keuze voor 500 gebruikers. Een /24 (254) is te klein en een /22 (1022) onnodig groot
Complexiteit	Eén VLAN per doelgroep houdt het beleid eenvoudig
Beheer	Duidelijke naamgeving in de YAML (office, guest, management) en per VLAN een voorspelbare prefix

4.3.5 Routering op een Linux-gateway (natgw) in plaats van in Faucet

Keuze. natgw is router, NAT-punt en firewall, met een subinterface per VLAN op a_core poort 4 (router-on-a-stick).

Alternatieven.

- Faucet-routering (faucet_vips): Faucet routeert per datapath, dus je zou per switch een gateway-instantie krijgen.
- Een gateway per gebouw: robuuster, maar vraagt twee NAT-punten en dus meer publieke adressen dan het enkele /28 biedt.

Criterium	Afweging
Prestaties	Al het verkeer tussen VLAN's en naar buiten loopt door één Linux-host. Voor het prototype is dat voldoende. Voor hoge doorvoer is een hardware-router of gedistribueerde routering beter. Verkeer binnen een VLAN blijft buiten de gateway
Schaalbaarheid	De gateway is de bottleneck voor intern-routed en uitgaand verkeer
Complexiteit	Routering, NAT en firewall staan op één plek en zijn eenvoudig te testen met iptables-tellers
Beheer	Eén beleidspunt voor inter-VLAN-verkeer. Dat is overzichtelijker dan regels verdeeld over zeven switches

Nadeel. natgw is een single point of failure. Een tweede gateway met VRRP zou dit oplossen.

4.3.6 Stateful firewall met conntrack op natgw

Keuze. iptables en ip6tables met connection tracking.

Alternatieven.

- Stateless ACL's in Faucet: de eerste versie blokkeerde daarmee ook het terugverkeer (3.3.5).
- Conntrack in Open vSwitch via OpenFlow-extensies: technisch mogelijk, maar Faucet programmeert dit niet. Je zou een eigen controllerapplicatie moeten schrijven.

Criterium	Afweging
Prestaties	Netfilter verwerkt verkeer in de kernel. Voor dit prototype geen knelpunt
Schaalbaarheid	De conntrack-tabel houdt 500 gelijktijdige verbindingen zonder probleem aan
Complexiteit	Standaard Linux-tooling met veel documentatie
Beheer	De tellers (iptables -L -n -v) laten per regel zien wat er gebeurt

4.3.7 NAT/PAT op één publiek adres

Keuze. SNAT van alle interne IPv4-adressen naar 198.51.100.2.

Alternatief. 1-op-1 NAT met meerdere publieke adressen: het /28 heeft er maar 14 bruikbare, dus dat dekt nooit 500 gebruikers.

Criterium	Afweging
Prestaties	Vertaling in de kernel via conntrack
Schaalbaarheid	Eén adres draagt in de orde van tienduizenden verbindingen per bestemming
Complexiteit	Eén regel
Beheer	De overige adressen in het /28 blijven beschikbaar voor servers

Voor IPv6 gebruik ik geen NAT. De ruime adresruimte maakt dat onnodig en het houdt de verbindingen end-to-end. De firewall bepaalt het beleid.

4.3.8 Rate limiting met een OpenFlow-meter

Keuze. Meter guest-rate-limit van 1 Mbit/s, aangeroepen vanuit de gast-ACL.

Alternatieven. tc (traffic control) op de hostinterface of een queue op de switchpoort.

Criterium	Afweging
Prestaties	De limiet wordt in de datapath toegepast, voordat het verkeer het netwerk in gaat
Schaalbaarheid	Een meter geldt per switch voor alle flows die ernaar verwijzen. Met één gastpoort per switch werkt dit per poort. Met meer gastpoorten op één switch delen ze dezelfde 1 Mbit/s en is een meter per poort nodig
Complexiteit	Vier regels in de YAML
Beheer	Centraal ingesteld en zichtbaar met dump-meters en meter-stats. tc moet op elke host worden ingesteld

4.3.9 Port security met een ACL op bron-MAC

Keuze. Per managementpoort één toegestaan MAC-adres en daaronder een drop-regel.

Alternatieven. De Faucet-optie max_hosts per poort, of 802.1X.

Criterium	Afweging
Prestaties	Eén flow-regel per poort, verwaarloosbare belasting
Schaalbaarheid	Statisch beheer van MAC's is bij veel poorten arbeidsintensief. Bij grotere aantallen is max_hosts of 802.1X beter
Complexiteit	Eenvoudig te begrijpen en te testen
Beheer	Elke poort heeft een eigen ACL-blok. Dat levert veel herhaling in de YAML op. Met YAML-ankers of sjablonen is dat te beperken

Nadeel. MAC-adressen zijn te vervalsen. Port security is een eerste hindernis en geen volledige authenticatie.

4.3.10 Samenvatting

Keuze	Prestaties	Schaalbaarheid	Complexiteit	Beheer
Core/access-topologie	goed	goed	laag	goed
Eén controller	goed	voldoende	laag	goed
Faucet	goed	goed	laag	goed
/23-adressering	voldoende	voldoende	laag	goed
Gateway natgw	beperkt (bottleneck)	beperkt	laag	goed
Stateful firewall op natgw	voldoende	voldoende	laag	goed
PAT op één adres	goed	goed	laag	goed
Meter voor rate limiting	goed	voldoende	laag	goed
Port security via ACL	goed	beperkt	laag	beperkt

4.3.11 Beperkingen en aanbevelingen voor productie

- natgw en de controller zijn enkelvoudig uitgevoerd. Redundantie kan met VRRP en een controllercluster.
- Gebouw B bereikt het internet via de darkfiber en a_core, omdat het ene /28 één NAT-punt oplegt. Met een tweede publiek blok kan gebouw B een eigen uplink krijgen.
- De 500-gebruikerstest gebruikt één bronhost met één MAC. Dat bewijst NAT en conntrack, niet het leren van 500 MAC-adressen.
- Hosts hebben statische IPv6-adressen. Productie vraagt SLAAC of DHCPv6.
- De gateway accepteert al het interne verkeer op INPUT. In productie beperk je dat tot ICMP en de benodigde diensten.

4.4 Functioneel bewijs

4.4.1 Flow entries, meter en firewall (figuur 0 tot 15)

Figuur 0 tot en met 15 zijn dumps uit Mininet. Ze tonen de flow entries per functionaliteit (zie 4.2), de meter en de iptables-regels van natgw. Boven elke figuur staat welke sectie ervan bij hoort en wat je moet zien.

```
$ ovs-ofctl -O OpenFlow13 dump-flows <switch> | grep -o 'table=[0-9]*' | sort | uniq -c
a_core table=0: 13   table=1: 17   table=2: 16   table=3: 19
a_f1   table=0: 21   table=1: 7   table=2: 17   table=3: 16   table=4: 19
a_f2   table=0: 21   table=1: 7   table=2: 12   table=3: 16   table=4: 19
b_core table=0: 13   table=1: 12   table=2: 15   table=3: 19
b_f1   table=0: 21   table=1: 7   table=2: 12   table=3: 15   table=4: 19
b_f2   table=0: 21   table=1: 7   table=2: 12   table=3: 15   table=4: 19
b_f3   table=0: 21   table=1: 7   table=2: 12   table=3: 15   table=4: 19
```

```
 FIGUUR 1   |   Sectie 1: VLAN-segmentatie en trunking
 Wat je moet zien : poort 2, 3, 4: push_vlan + set_field 4196, 4296, 4396; poort 1 (trunk): dl_vlan 100, 200, 300; p
riority=0 drop
 Onderschrift     : Tabel 1 van a_f1: een hostpoort geeft ongetagd verkeer het VLAN van de poort, de trunk accepteer
t alleen VLAN 100, 200 en 300.
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_f1 table=1
 table=1, n_packets=132400, n_bytes=9269876, priority=4096,in_port="a_f1-eth1",dl_vlan=100 actions=goto_table:2
 table=1, n_packets=787, n_bytes=57846, priority=4096,in_port="a_f1-eth1",dl_vlan=200 actions=goto_table:2
 table=1, n_packets=110, n_bytes=9704, priority=4096,in_port="a_f1-eth1",dl_vlan=300 actions=goto_table:2
 table=1, n_packets=186978, n_bytes=8213385472, priority=4096,in_port="a_f1-eth2",vlan_tci=0x0000/0x1fff actions=pus
h_vlan:0x8100,set_field:4196->vlan_vid,goto_table:2
 table=1, n_packets=1004, n_bytes=1689212, priority=4096,in_port="a_f1-eth3",vlan_tci=0x0000/0x1fff actions=push_vla
n:0x8100,set_field:4296->vlan_vid,goto_table:2
 table=1, n_packets=71, n_bytes=5698, priority=4096,in_port="a_f1-eth4",vlan_tci=0x0000/0x1fff actions=push_vlan:0x8
100,set_field:4396->vlan_vid,goto_table:2
 table=1, n_packets=0, n_bytes=0, priority=0 actions=drop
```

```
 FIGUUR 2   |   Sectie 4: Gastisolatie
 Wat je moet zien : arp en icmp6 (135, 136) toegestaan; drops naar 10.0.200.0/24, 10.0.100.0/23, 10.0.30.0/24 en IPv
6-prefixen; meter:1 naar gateway-MAC; catch-all drop; n_packets > 0
 Onderschrift     : Tabel 0 van a_f1, gastpoort, gesorteerd op prioriteit: de drops staan boven de toelating naar he
t gateway-MAC.
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_f1 table=0 | grep -E 'in_port="a_f1-eth3"([, ]|$)'
 table=0, n_packets=16, n_bytes=672, priority=20480,arp,in_port="a_f1-eth3" actions=goto_table:1
 table=0, n_packets=4, n_bytes=344, priority=20479,icmp6,in_port="a_f1-eth3",icmp_type=135 actions=goto_table:1
 table=0, n_packets=4, n_bytes=312, priority=20478,icmp6,in_port="a_f1-eth3",icmp_type=136 actions=goto_table:1
 table=0, n_packets=0, n_bytes=0, priority=20477,ip,in_port="a_f1-eth3",dl_dst=02:00:00:00:fe:01,nw_dst=10.0.200.254
 actions=meter:1,goto_table:1
 table=0, n_packets=3, n_bytes=294, priority=20476,ip,in_port="a_f1-eth3",nw_dst=10.0.200.0/24 actions=drop
 table=0, n_packets=3, n_bytes=294, priority=20475,ip,in_port="a_f1-eth3",nw_dst=10.0.100.0/23 actions=drop
 table=0, n_packets=3, n_bytes=294, priority=20474,ip,in_port="a_f1-eth3",nw_dst=10.0.30.0/24 actions=drop
 table=0, n_packets=979, n_bytes=1687726, priority=20473,ip,in_port="a_f1-eth3",dl_dst=02:00:00:00:fe:01 actions=met
er:1,goto_table:1
 table=0, n_packets=0, n_bytes=0, priority=20472,ipv6,in_port="a_f1-eth3",dl_dst=02:00:00:00:fe:01,ipv6_dst=2001:db8
:200::1 actions=meter:1,goto_table:1
 table=0, n_packets=3, n_bytes=354, priority=20471,ipv6,in_port="a_f1-eth3",ipv6_dst=2001:db8:200::/64 actions=drop
 table=0, n_packets=0, n_bytes=0, priority=20470,ipv6,in_port="a_f1-eth3",ipv6_dst=2001:db8:100::/64 actions=drop
 table=0, n_packets=0, n_bytes=0, priority=20469,ipv6,in_port="a_f1-eth3",ipv6_dst=2001:db8:300::/64 actions=drop
 table=0, n_packets=3, n_bytes=354, priority=20468,ipv6,in_port="a_f1-eth3",dl_dst=02:00:00:00:fe:01 actions=meter:1
,goto_table:1
 table=0, n_packets=11, n_bytes=890, priority=20467,in_port="a_f1-eth3" actions=drop
```

```
 FIGUUR 3   |   Sectie 5: Rate limiting
 Wat je moet zien : meter 1 met rate=1000 burst_size=100; drie gastregels met actions=meter:1
 Onderschrift     : Meter 1 (1 Mbit/s, drop-band) en de flow entries in tabel 0 die ernaar verwijzen.
$ ovs-ofctl -O OpenFlow13 dump-meters a_f1
OFPST_METER_CONFIG reply (OF1.3) (xid=0x2):
meter=1 kbps burst stats bands=
type=drop rate=1000 burst_size=100

$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_f1 table=0 | grep -E 'meter:1'
 table=0, n_packets=0, n_bytes=0, priority=20477,ip,in_port="a_f1-eth3",dl_dst=02:00:00:00:fe:01,nw_dst=10.0.200.254
 actions=meter:1,goto_table:1
 table=0, n_packets=980, n_bytes=1687824, priority=20473,ip,in_port="a_f1-eth3",dl_dst=02:00:00:00:fe:01 actions=met
er:1,goto_table:1
 table=0, n_packets=0, n_bytes=0, priority=20472,ipv6,in_port="a_f1-eth3",dl_dst=02:00:00:00:fe:01,ipv6_dst=2001:db8
:200::1 actions=meter:1,goto_table:1
 table=0, n_packets=3, n_bytes=354, priority=20468,ipv6,in_port="a_f1-eth3",dl_dst=02:00:00:00:fe:01 actions=meter:1
,goto_table:1
```

```
================================================================================
 FIGUUR 4   |   Sectie 5: Rate limiting
================================================================================
 Wat je moet zien : packet_in_count en byte_in_count; bij bands de packet_count (gedropte pakketten) groter dan 0
 Onderschrift     : Statistiek van meter 1: pakketten in de drop-band bewijzen dat de limiet is toegepast.
--------------------------------------------------------------------------------
$ ovs-ofctl -O OpenFlow13 meter-stats a_f1
OFPST_METER reply (OF1.3) (xid=0x2):
meter:1 flow_count:4 packet_in_count:984 byte_in_count:1688276 duration:407.110s bands:
0: packet_count:145 byte_count:320890
```

```
================================================================================
 FIGUUR 5   |   Sectie 6: Port security
================================================================================
 Wat je moet zien : allow op dl_src=02:00:00:00:30:11 (prioriteit 20480) en daaronder een drop (20479) met n_packets
 > 0
 Onderschrift     : Tabel 0 van a_f1, managementpoort: alleen het geregistreerde MAC-adres mag door, al het andere w
ordt gedropt.
--------------------------------------------------------------------------------
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_f1 table=0 | grep -E 'in_port="a_f1-eth4"([, ]|$)'
 table=0, n_packets=79, n_bytes=6314, priority=20480,in_port="a_f1-eth4",dl_src=02:00:00:00:30:11 actions=goto_table
:1
 table=0, n_packets=3, n_bytes=126, priority=20479,in_port="a_f1-eth4" actions=drop
```

```
================================================================================
 FIGUUR 6   |   Sectie 2: MAC-learning en forwarding
================================================================================
 Wat je moet zien : bovenaan beveiligingsregels (drop dl_src ff:ff.., 0e:00.., 0x9000); geleerde bron-MAC's (priorit
y 8191); onderaan CONTROLLER:96
 Onderschrift     : Tabel 2 van a_f1: Faucet leert bron-MAC's per poort en VLAN; onbekende bron-MAC's gaan naar de c
ontroller.
--------------------------------------------------------------------------------
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_f1 table=2
 table=2, n_packets=0, n_bytes=0, priority=20490,dl_type=0x9000 actions=drop
 table=2, n_packets=0, n_bytes=0, priority=20480,dl_src=ff:ff:ff:ff:ff:ff actions=drop
 table=2, n_packets=0, n_bytes=0, priority=20480,dl_src=0e:00:00:00:00:01 actions=drop
 table=2, n_packets=0, n_bytes=0, priority=8191,in_port="a_f1-eth1",dl_vlan=100,dl_src=00:00:00:00:00:07 actions=got
o_table:3
 table=2, n_packets=27, n_bytes=2114, priority=8191,in_port="a_f1-eth4",dl_vlan=300,dl_src=02:00:00:00:30:11 actions
=goto_table:3
 table=2, n_packets=0, n_bytes=0, priority=8191,in_port="a_f1-eth1",dl_vlan=300,dl_src=02:00:00:00:30:22 actions=got
o_table:3
 table=2, n_packets=0, n_bytes=0, priority=8191,in_port="a_f1-eth1",dl_vlan=100,dl_src=00:00:00:00:00:0a actions=got
o_table:3
 table=2, n_packets=18, n_bytes=1556, priority=8191,in_port="a_f1-eth1",dl_vlan=200,dl_src=02:00:00:00:fe:01 actions
=goto_table:3
 table=2, n_packets=20, n_bytes=1648, priority=8191,in_port="a_f1-eth1",dl_vlan=300,dl_src=02:00:00:00:fe:01 actions
=goto_table:3
 table=2, n_packets=19, n_bytes=1526, priority=8191,in_port="a_f1-eth2",dl_vlan=100,dl_src=00:00:00:00:00:01 actions
=goto_table:3
 table=2, n_packets=19, n_bytes=1602, priority=8191,in_port="a_f1-eth1",dl_vlan=100,dl_src=02:00:00:00:fe:01 actions
=goto_table:3
 table=2, n_packets=0, n_bytes=0, priority=8191,in_port="a_f1-eth1",dl_vlan=300,dl_src=02:00:00:00:30:12 actions=got
o_table:3
 table=2, n_packets=13, n_bytes=1050, priority=8191,in_port="a_f1-eth3",dl_vlan=200,dl_src=00:00:00:00:00:02 actions
=goto_table:3
 table=2, n_packets=10, n_bytes=924, priority=4096,dl_vlan=100 actions=CONTROLLER:96,goto_table:3
 table=2, n_packets=6, n_bytes=484, priority=4096,dl_vlan=200 actions=CONTROLLER:96,goto_table:3
 table=2, n_packets=12, n_bytes=1064, priority=4096,dl_vlan=300 actions=CONTROLLER:96,goto_table:3
 table=2, n_packets=0, n_bytes=0, priority=0 actions=goto_table:3
```

```
================================================================================
 FIGUUR 7   |   Sectie 2: MAC-learning en forwarding
================================================================================
 Wat je moet zien : pop_vlan + output naar hostpoorten; output zonder pop_vlan naar trunk (poort 1); priority=0 goto
_table:4
 Onderschrift     : Tabel 3 van a_f1: doorsturen op doel-MAC. Hostpoorten krijgen ongetagd verkeer, de trunk behoudt
 de tag.
--------------------------------------------------------------------------------
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_f1 table=3
 table=3, n_packets=70, n_bytes=5852, priority=8192,dl_vlan=300,dl_dst=02:00:00:00:30:11 actions=pop_vlan,output:"a_
f1-eth4"
 table=3, n_packets=4, n_bytes=336, priority=8192,dl_vlan=100,dl_dst=00:00:00:00:00:0d actions=output:"a_f1-eth1"
 table=3, n_packets=132359, n_bytes=9266038, priority=8192,dl_vlan=100,dl_dst=00:00:00:00:00:01 actions=pop_vlan,out
put:"a_f1-eth2"
 table=3, n_packets=186974, n_bytes=8213385100, priority=8192,dl_vlan=100,dl_dst=02:00:00:00:fe:01 actions=output:"a
_f1-eth1"
 table=3, n_packets=1007, n_bytes=1689526, priority=8192,dl_vlan=200,dl_dst=02:00:00:00:fe:01 actions=output:"a_f1-e
th1"
 table=3, n_packets=68, n_bytes=5488, priority=8192,dl_vlan=300,dl_dst=02:00:00:00:fe:01 actions=output:"a_f1-eth1"
 table=3, n_packets=788, n_bytes=57536, priority=8192,dl_vlan=200,dl_dst=00:00:00:00:00:02 actions=pop_vlan,output:"
a_f1-eth3"
 table=3, n_packets=3, n_bytes=242, priority=8192,dl_vlan=200,dl_dst=00:00:00:00:00:05 actions=output:"a_f1-eth1"
 table=3, n_packets=0, n_bytes=0, priority=8192,dl_vlan=300,dl_dst=02:00:00:00:30:12 actions=output:"a_f1-eth1"
 table=3, n_packets=143, n_bytes=12946, priority=0 actions=goto_table:4
```

```
================================================================================
 FIGUUR 8   |   Sectie 3: Flooding, ARP en IPv6 ND
================================================================================
 Wat je moet zien : broadcast (ff:ff..) en IPv6-multicast (33:33..) per VLAN naar trunk en hostpoort; drops voor 01:
00:0c.. en 01:80:c2..
 Onderschrift     : Tabel 4 van a_f1: broadcast en multicast blijven binnen het eigen VLAN; beheerverkeer van andere
 apparatuur wordt gedropt.
--------------------------------------------------------------------------------
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_f1 table=4
 table=4, n_packets=2, n_bytes=88, priority=8240,dl_vlan=100,dl_dst=ff:ff:ff:ff:ff:ff actions=output:"a_f1-eth1",pop
_vlan,output:"a_f1-eth2"
 table=4, n_packets=2, n_bytes=84, priority=8240,dl_vlan=200,dl_dst=ff:ff:ff:ff:ff:ff actions=output:"a_f1-eth1",pop
_vlan,output:"a_f1-eth3"
 table=4, n_packets=2, n_bytes=84, priority=8240,dl_vlan=300,dl_dst=ff:ff:ff:ff:ff:ff actions=output:"a_f1-eth1",pop
_vlan,output:"a_f1-eth4"
 table=4, n_packets=0, n_bytes=0, priority=8240,dl_dst=01:00:0c:cc:cc:cc actions=drop
 table=4, n_packets=0, n_bytes=0, priority=8240,dl_dst=01:00:0c:cc:cc:cd actions=drop
 table=4, n_packets=0, n_bytes=0, priority=8236,dl_dst=01:80:c2:00:00:00/ff:ff:ff:ff:ff:f0 actions=drop
 table=4, n_packets=0, n_bytes=0, priority=8216,dl_vlan=100,dl_dst=01:80:c2:00:00:00/ff:ff:ff:00:00:00 actions=outpu
t:"a_f1-eth1",pop_vlan,output:"a_f1-eth2"
 table=4, n_packets=0, n_bytes=0, priority=8216,dl_vlan=100,dl_dst=01:00:5e:00:00:00/ff:ff:ff:00:00:00 actions=outpu
t:"a_f1-eth1",pop_vlan,output:"a_f1-eth2"
 table=4, n_packets=0, n_bytes=0, priority=8216,dl_vlan=200,dl_dst=01:80:c2:00:00:00/ff:ff:ff:00:00:00 actions=outpu
t:"a_f1-eth1",pop_vlan,output:"a_f1-eth3"
 table=4, n_packets=0, n_bytes=0, priority=8216,dl_vlan=200,dl_dst=01:00:5e:00:00:00/ff:ff:ff:00:00:00 actions=outpu
t:"a_f1-eth1",pop_vlan,output:"a_f1-eth3"
 table=4, n_packets=0, n_bytes=0, priority=8216,dl_vlan=300,dl_dst=01:80:c2:00:00:00/ff:ff:ff:00:00:00 actions=outpu
t:"a_f1-eth1",pop_vlan,output:"a_f1-eth4"
 table=4, n_packets=0, n_bytes=0, priority=8216,dl_vlan=300,dl_dst=01:00:5e:00:00:00/ff:ff:ff:00:00:00 actions=outpu
t:"a_f1-eth1",pop_vlan,output:"a_f1-eth4"
 table=4, n_packets=63, n_bytes=5626, priority=8208,dl_vlan=100,dl_dst=33:33:00:00:00:00/ff:ff:00:00:00:00 actions=o
utput:"a_f1-eth1",pop_vlan,output:"a_f1-eth2"
 table=4, n_packets=9, n_bytes=1186, priority=8208,dl_vlan=200,dl_dst=33:33:00:00:00:00/ff:ff:00:00:00:00 actions=ou
tput:"a_f1-eth1",pop_vlan,output:"a_f1-eth3"
 table=4, n_packets=63, n_bytes=5730, priority=8208,dl_vlan=300,dl_dst=33:33:00:00:00:00/ff:ff:00:00:00:00 actions=o
utput:"a_f1-eth1",pop_vlan,output:"a_f1-eth4"
 table=4, n_packets=0, n_bytes=0, priority=8192,dl_vlan=100 actions=output:"a_f1-eth1",pop_vlan,output:"a_f1-eth2"
 table=4, n_packets=2, n_bytes=148, priority=8192,dl_vlan=200 actions=output:"a_f1-eth1",pop_vlan,output:"a_f1-eth3"
 table=4, n_packets=0, n_bytes=0, priority=8192,dl_vlan=300 actions=output:"a_f1-eth1",pop_vlan,output:"a_f1-eth4"
 table=4, n_packets=0, n_bytes=0, priority=0 actions=drop
```

```
================================================================================
  FIGUUR 9   |   Sectie 7: Verkeer tussen verdiepingen, gebouwen en naar de gateway
================================================================================
  Wat je moet zien : tabel 0: dl_vlan 100, 200, 300 per poort; tabel 2: dl_dst=02:00:00:00:fe:01 -> poort 4 (natgw),
 hosts in gebouw B -> poort 3 (darkfiber)
  Onderschrift     : a_core heeft geen ACL's: tabel 0 = VLAN, 1 = bron-MAC, 2 = doel-MAC, 3 = flooding. Alle uitgange
 n zijn trunks, dus de tag blijft.
--------------------------------------------------------------------------------
 $ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_core table=0
  table=0, n_packets=186991, n_bytes=8214134430, priority=4096,in_port="a_core-eth1",dl_vlan=100 actions=goto_table:1
  table=0, n_packets=872, n_bytes=1372860, priority=4096,in_port="a_core-eth1",dl_vlan=200 actions=goto_table:1
  table=0, n_packets=83, n_bytes=6982, priority=4096,in_port="a_core-eth1",dl_vlan=300 actions=goto_table:1
  table=0, n_packets=11, n_bytes=934, priority=4096,in_port="a_core-eth2",dl_vlan=100 actions=goto_table:1
  table=0, n_packets=5, n_bytes=398, priority=4096,in_port="a_core-eth2",dl_vlan=200 actions=goto_table:1
  table=0, n_packets=11, n_bytes=934, priority=4096,in_port="a_core-eth2",dl_vlan=300 actions=goto_table:1
  table=0, n_packets=41, n_bytes=3494, priority=4096,in_port="a_core-eth3",dl_vlan=100 actions=goto_table:1
  table=0, n_packets=0, n_bytes=0, priority=4096,in_port="a_core-eth3",dl_vlan=200 actions=goto_table:1
  table=0, n_packets=33, n_bytes=2802, priority=4096,in_port="a_core-eth3",dl_vlan=300 actions=goto_table:1
  table=0, n_packets=132361, n_bytes=9266494, priority=4096,in_port="a_core-eth4",dl_vlan=100 actions=goto_table:1
  table=0, n_packets=795, n_bytes=58550, priority=4096,in_port="a_core-eth4",dl_vlan=200 actions=goto_table:1
  table=0, n_packets=78, n_bytes=6968, priority=4096,in_port="a_core-eth4",dl_vlan=300 actions=goto_table:1
  table=0, n_packets=7, n_bytes=710, priority=0 actions=drop

 $ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_core table=2
  table=2, n_packets=71, n_bytes=5954, priority=8192,dl_vlan=300,dl_dst=02:00:00:00:30:11 actions=output:"a_core-eth1
 "
  table=2, n_packets=132360, n_bytes=9266140, priority=8192,dl_vlan=100,dl_dst=00:00:00:00:00:01 actions=output:"a_co
 re-eth1"
  table=2, n_packets=186975, n_bytes=8214133098, priority=8192,dl_vlan=100,dl_dst=02:00:00:00:fe:01 actions=output:"a
 _core-eth4"
  table=2, n_packets=865, n_bytes=1372334, priority=8192,dl_vlan=200,dl_dst=02:00:00:00:fe:01 actions=output:"a_core-
 eth4"
  table=2, n_packets=69, n_bytes=5862, priority=8192,dl_vlan=300,dl_dst=02:00:00:00:fe:01 actions=output:"a_core-eth4
 "
  table=2, n_packets=791, n_bytes=57786, priority=8192,dl_vlan=200,dl_dst=00:00:00:00:00:02 actions=output:"a_core-et
 h1"
  table=2, n_packets=3, n_bytes=254, priority=8192,dl_vlan=200,dl_dst=00:00:00:00:00:05 actions=output:"a_core-eth2"
  table=2, n_packets=0, n_bytes=0, priority=8192,dl_vlan=300,dl_dst=02:00:00:00:30:12 actions=output:"a_core-eth2"
  table=2, n_packets=143, n_bytes=13066, priority=0 actions=goto_table:3
```

```
FIGUUR 10   |   Sectie 7: Darkfiber
Wat je moet zien : in_port 4 (darkfiber) met dl_vlan 100, 200 en 300 -> goto_table:1; priority=0 drop
Onderschrift     : Tabel 0 van b_core: de darkfiber vervoert alle drie de VLAN's getagd.
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows b_core table=0
 table=0, n_packets=12, n_bytes=1048, priority=4096,in_port="b_core-eth1",dl_vlan=100 actions=goto_table:1
 table=0, n_packets=0, n_bytes=0, priority=4096,in_port="b_core-eth1",dl_vlan=200 actions=goto_table:1
 table=0, n_packets=11, n_bytes=934, priority=4096,in_port="b_core-eth1",dl_vlan=300 actions=goto_table:1
 table=0, n_packets=12, n_bytes=1024, priority=4096,in_port="b_core-eth2",dl_vlan=100 actions=goto_table:1
 table=0, n_packets=0, n_bytes=0, priority=4096,in_port="b_core-eth2",dl_vlan=200 actions=goto_table:1
 table=0, n_packets=11, n_bytes=934, priority=4096,in_port="b_core-eth2",dl_vlan=300 actions=goto_table:1
 table=0, n_packets=17, n_bytes=1422, priority=4096,in_port="b_core-eth3",dl_vlan=100 actions=goto_table:1
 table=0, n_packets=0, n_bytes=0, priority=4096,in_port="b_core-eth3",dl_vlan=200 actions=goto_table:1
 table=0, n_packets=11, n_bytes=934, priority=4096,in_port="b_core-eth3",dl_vlan=300 actions=goto_table:1
 table=0, n_packets=33, n_bytes=3018, priority=4096,in_port="b_core-eth4",dl_vlan=100 actions=goto_table:1
 table=0, n_packets=13, n_bytes=1434, priority=4096,in_port="b_core-eth4",dl_vlan=200 actions=goto_table:1
 table=0, n_packets=32, n_bytes=3068, priority=4096,in_port="b_core-eth4",dl_vlan=300 actions=goto_table:1
 table=0, n_packets=0, n_bytes=0, priority=0 actions=drop
```

```
FIGUUR 11   |   Sectie 8: Stateful firewall (IPv4)
Wat je moet zien : policy DROP met teller; ctstate INVALID drop; ESTABLISHED,RELATED accept met hoge teller; ctstat
e NEW alleen van binnen naar buiten
Onderschrift     : Firewall op natgw: alleen intern geinitieerde verbindingen komen door, terugverkeer volgt via ES
TABLISHED.
$ natgw iptables -L FORWARD -n -v
Chain FORWARD (policy DROP 6 packets, 504 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DROP       all  --  *      *       0.0.0.0/0            0.0.0.0/0           ctstate INVALID
 321K 8219M ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0           ctstate RELATED,ESTABLISHED
    2   120 ACCEPT     all  --  natgw-eth0.100 natgw-eth1  0.0.0.0/0            0.0.0.0/0           ctstate NEW
    4   288 ACCEPT     all  --  natgw-eth0.200 natgw-eth1  0.0.0.0/0            0.0.0.0/0           ctstate NEW
    1    84 ACCEPT     all  --  natgw-eth0.300 natgw-eth1  0.0.0.0/0            0.0.0.0/0           ctstate NEW
    1    84 ACCEPT     all  --  natgw-eth0.300 natgw-eth0.100  0.0.0.0/0            0.0.0.0/0           ctstate NEW
```

```
FIGUUR 12   |   Sectie 8: Stateful firewall (IPv6)
Wat je moet zien : dezelfde opbouw als IPv4: policy DROP met teller, ESTABLISHED,RELATED, ctstate NEW van binnen na
ar buiten
Onderschrift     : De IPv6-firewall op natgw: geen NAT, maar dezelfde stateful regels.
$ natgw ip6tables -L FORWARD -n -v
Chain FORWARD (policy DROP 3 packets, 312 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DROP       all      *      *       ::/0                 ::/0                 ctstate INVALID
    5   520 ACCEPT     all      *      *       ::/0                 ::/0                 ctstate RELATED,ESTABLISHED
    0     0 ACCEPT     all      natgw-eth0.100 natgw-eth1  ::/0                ::/0                 ctstate NEW
    1   104 ACCEPT     all      natgw-eth0.200 natgw-eth1  ::/0                ::/0                 ctstate NEW
    0     0 ACCEPT     all      natgw-eth0.300 natgw-eth1  ::/0                ::/0                 ctstate NEW
    0     0 ACCEPT     all      natgw-eth0.300 natgw-eth0.100  ::/0                ::/0                 ctstate NEW
```

```
FIGUUR 13   |   Sectie 8: NAT
Wat je moet zien : SNAT-regel van 10.0.0.0/8 naar 198.51.100.2 met pkts en bytes
Onderschrift     : PAT op natgw: alle interne IPv4-hosts delen het publieke adres 198.51.100.2.
$ natgw iptables -t nat -L POSTROUTING -n -v
Chain POSTROUTING (policy ACCEPT 1 packets, 84 bytes)
 pkts bytes target     prot opt in     out     source               destination
    7   492 SNAT       all  --  *      natgw-eth1  10.0.0.0/8           0.0.0.0/0           to:198.51.100.2
```

```
================================================================================
 FIGUUR 14   |   Sectie 9: Pad van een pakket, gast naar ISP (a_f1)
================================================================================
 Wat je moet zien : stap 1 tabel 0 meter:1; stap 2 tabel 1 push_vlan 4296; stap 3 tabel 2 geleerd MAC; stap 4 tabel
3 output naar trunk; extra: gast naar gast = drop
 Onderschrift     : Het pad van een gastpakket door de tabellen van a_f1, en het geblokkeerde pad naar een andere ga
st.
--------------------------------------------------------------------------------
# Stap 1 (tabel 0): toegestaan naar het gateway-MAC, via de meter
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_f1 table=0 | grep -E 'in_port="a_f1-eth3"([, ]|$).*d
l_dst=02:00:00:00:fe:01'
 table=0, n_packets=0, n_bytes=0, priority=20477,ip,in_port="a_f1-eth3",dl_dst=02:00:00:00:fe:01,nw_dst=10.0.200.254
 actions=meter:1,goto_table:1
 table=0, n_packets=990, n_bytes=1688804, priority=20473,ip,in_port="a_f1-eth3",dl_dst=02:00:00:00:fe:01 actions=met
er:1,goto_table:1
 table=0, n_packets=0, n_bytes=0, priority=20472,ipv6,in_port="a_f1-eth3",dl_dst=02:00:00:00:fe:01,ipv6_dst=2001:db8
:200::1 actions=meter:1,goto_table:1
 table=0, n_packets=3, n_bytes=354, priority=20468,ipv6,in_port="a_f1-eth3",dl_dst=02:00:00:00:fe:01 actions=meter:1
,goto_table:1

# Stap 2 (tabel 1): ongetagd verkeer krijgt VLAN 200
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_f1 table=1 | grep -E 'in_port="a_f1-eth3"([, ]|$)'
 table=1, n_packets=1023, n_bytes=1690738, priority=4096,in_port="a_f1-eth3",vlan_tci=0x0000/0x1fff actions=push_vla
n:0x8100,set_field:4296->vlan_vid,goto_table:2

# Stap 3 (tabel 2): bron-MAC van a1_gst is geleerd
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_f1 table=2 | grep -E 'in_port="a_f1-eth3"([, ]|$).*d
l_src=00:00:00:00:00:02'
 table=2, n_packets=24, n_bytes=1904, priority=8191,in_port="a_f1-eth3",dl_vlan=200,dl_src=00:00:00:00:00:02 actions
=goto_table:3

# Stap 4 (tabel 3): doorsturen naar de gateway via de trunk
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_f1 table=3 | grep -E 'dl_vlan=200,dl_dst=02:00:00:00
:fe:01'
 table=3, n_packets=1016, n_bytes=1690240, priority=8192,dl_vlan=200,dl_dst=02:00:00:00:fe:01 actions=output:"a_f1-e
th1"

# Extra: gast naar een andere gast wordt in tabel 0 gedropt
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_f1 table=0 | grep -E 'in_port="a_f1-eth3"([, ]|$).*n
w_dst=10.0.200.0/24'
 table=0, n_packets=3, n_bytes=294, priority=20476,ip,in_port="a_f1-eth3",nw_dst=10.0.200.0/24 actions=drop
```

```
================================================================================
 FIGUUR 15   |   Sectie 9: Pad van een pakket, gast naar ISP (a_core)
================================================================================
 Wat je moet zien : stap 5 tabel 0 in_port 1 + dl_vlan=200; stap 6 tabel 2 naar poort 4 (natgw); terugweg: dl_dst a1
_gst -> poort 1, daarna pop_vlan op a_f1
 Onderschrift     : Het pad op de core en de terugweg naar de gast.
--------------------------------------------------------------------------------
# Stap 5 (a_core, tabel 0): komt binnen van a_f1 met VLAN 200
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_core table=0 | grep -E 'in_port="a_core-eth1"([, ]|$
).*dl_vlan=200'
 table=0, n_packets=879, n_bytes=1373462, priority=4096,in_port="a_core-eth1",dl_vlan=200 actions=goto_table:1

# Stap 6 (a_core, tabel 2): naar natgw (poort 4)
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_core table=2 | grep -E 'dl_vlan=200,dl_dst=02:00:00:
00:fe:01'
 table=2, n_packets=872, n_bytes=1372936, priority=8192,dl_vlan=200,dl_dst=02:00:00:00:fe:01 actions=output:"a_core-
eth4"

# Terugweg (a_core, tabel 2): naar a1_gst via poort 1
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_core table=2 | grep -E 'dl_vlan=200,dl_dst=00:00:00:
00:00:02'
 table=2, n_packets=798, n_bytes=58388, priority=8192,dl_vlan=200,dl_dst=00:00:00:00:00:02 actions=output:"a_core-et
h1"

# Terugweg (a_f1, tabel 3): tag eraf, uit via de gastpoort
$ ovs-ofctl --names --rsort=priority -O OpenFlow13 dump-flows a_f1 table=3 | grep -E 'dl_vlan=200,dl_dst=00:00:00:00
:00:02'
 table=3, n_packets=798, n_bytes=58388, priority=8192,dl_vlan=200,dl_dst=00:00:00:00:00:02 actions=pop_vlan,output:"
a_f1-eth3"
```

4.4.2 Connectiviteit en bandbreedte (ping, ping6 en iperf)

Doel: aantonen dat elk netwerk zijn gateway en de ISP bereikt, dat gasten elkaar niet bereiken, dat IPv6 werkt en dat de rate limit alleen voor gasten geldt.

```
mininet> a1_emp ping -c 3 10.0.100.254
mininet> a1_gst ping -c 3 198.51.100.1
mininet> a1_gst ping -c 3 10.0.200.12
mininet> a1_emp ping6 -c 3 2001:db8:ffff::1
mininet> iperf a1_gst isp_a
mininet> iperf a1_emp isp_a
```

```
mininet> a1_emp ping -c 3 10.0.100.254
PING 10.0.100.254 (10.0.100.254) 56(84) bytes of data.
64 bytes from 10.0.100.254: icmp_seq=1 ttl=64 time=4.14 ms
64 bytes from 10.0.100.254: icmp_seq=2 ttl=64 time=0.095 ms
64 bytes from 10.0.100.254: icmp_seq=3 ttl=64 time=0.082 ms

--- 10.0.100.254 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2017ms
rtt min/avg/max/mdev = 0.082/1.438/4.138/1.908 ms
mininet> a1_gst ping -c 3 198.51.100.1
PING 198.51.100.1 (198.51.100.1) 56(84) bytes of data.
64 bytes from 198.51.100.1: icmp_seq=1 ttl=63 time=1.33 ms
64 bytes from 198.51.100.1: icmp_seq=2 ttl=63 time=0.151 ms
64 bytes from 198.51.100.1: icmp_seq=3 ttl=63 time=0.170 ms

--- 198.51.100.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2029ms
rtt min/avg/max/mdev = 0.151/0.551/1.333/0.552 ms
mininet> a1_gst ping -c 3 10.0.200.12
PING 10.0.200.12 (10.0.200.12) 56(84) bytes of data.

--- 10.0.200.12 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2052ms

mininet> a1_emp ping6 -c 3 2001:db8:ffff::1
PING 2001:db8:ffff::1(2001:db8:ffff::1) 56 data bytes
64 bytes from 2001:db8:ffff::1: icmp_seq=1 ttl=63 time=1.75 ms
64 bytes from 2001:db8:ffff::1: icmp_seq=2 ttl=63 time=0.099 ms
64 bytes from 2001:db8:ffff::1: icmp_seq=3 ttl=63 time=0.134 ms

--- 2001:db8:ffff::1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2014ms
rtt min/avg/max/mdev = 0.099/0.660/1.747/0.768 ms
mininet> iperf a1_gst isp_a
*** Iperf: testing TCP bandwidth between a1_gst and isp_a
*** Results: ['1.08 Mbits/sec', '1.51 Mbits/sec']
mininet> iperf a1_emp isp_a
*** Iperf: testing TCP bandwidth between a1_emp and isp_a
*** Results: ['14.6 Gbits/sec', '14.6 Gbits/sec']
mininet>
```

Gateway en ISP. a1_emp bereikt de gateway van VLAN 100 (10.0.100.254) en a1_gst bereikt de ISP (198.51.100.1), beide met 0% verlies. De TTL van 63 bij de ISP laat zien dat er één router (natgw) tussen zit.

Gastisolatie. De ping van a1_gst naar een andere gast (10.0.200.12) geeft 100% verlies. De ACL op de gastpoort dropt dit verkeer in tabel 0 (zie figuur 2 en 14).

IPv6. De ping6 van a1_emp naar 2001:db8:ffff::1 slaagt met 0% verlies. De TTL van 63 laat opnieuw de route via natgw zien, zonder NAT.

Rate limiting. iperf geeft voor de gast 1,08 en 1,51 Mbit/s en voor de werknemer 14,6 Gbit/s. De meter van 1 Mbit/s (figuur 3 en 4) beperkt dus alleen het gastennetwerk. De waarde van de werknemer is een softwaresnelheid en is alleen bedoeld als vergelijking.

4.4.3 Route naar buiten (traceroute)

Doel: aantonen dat verkeer van elk VLAN via natgw als eerste hop naar de ISP loopt.

```
mininet> a1_emp traceroute -n 198.51.100.1
mininet> a1_gst traceroute -n 198.51.100.1
```

```
traceroute to 198.51.100.1 (198.51.100.1), 30 hops max, 60 byte packets
 1  10.0.100.254  2.814 ms  2.814 ms  2.781 ms
 2  198.51.100.1  4.420 ms  4.459 ms  4.508 ms
mininet> a1_gst traceroute -n 198.51.100.1
traceroute to 198.51.100.1 (198.51.100.1), 30 hops max, 60 byte packets
 1  10.0.200.254  5.036 ms  5.162 ms  5.402 ms
 2  198.51.100.1  5.662 ms  5.712 ms  5.735 ms
```

De eerste traceroute (a1_emp, VLAN 100) heeft als eerste hop 10.0.100.254 en daarna de ISP 198.51.100.1. De tweede (a1_gst, VLAN 200) heeft als eerste hop 10.0.200.254 en dan de ISP. Elk VLAN gebruikt dus zijn eigen subinterface op natgw als gateway (router-on-a-stick, 3.2.2) en alles loopt via maar één router.

Vanaf de ISP naar binnen. Doel: aantonen dat nieuwe verbindingen van de ISP naar binnen worden tegengehouden. Hiervoor krijgt isp_a eerst een route naar het employee-netwerk via natgw.

```
mininet> isp_a ip route add 10.0.100.0/23 via 198.51.100.2
mininet> isp_a traceroute 10.0.100.11
```

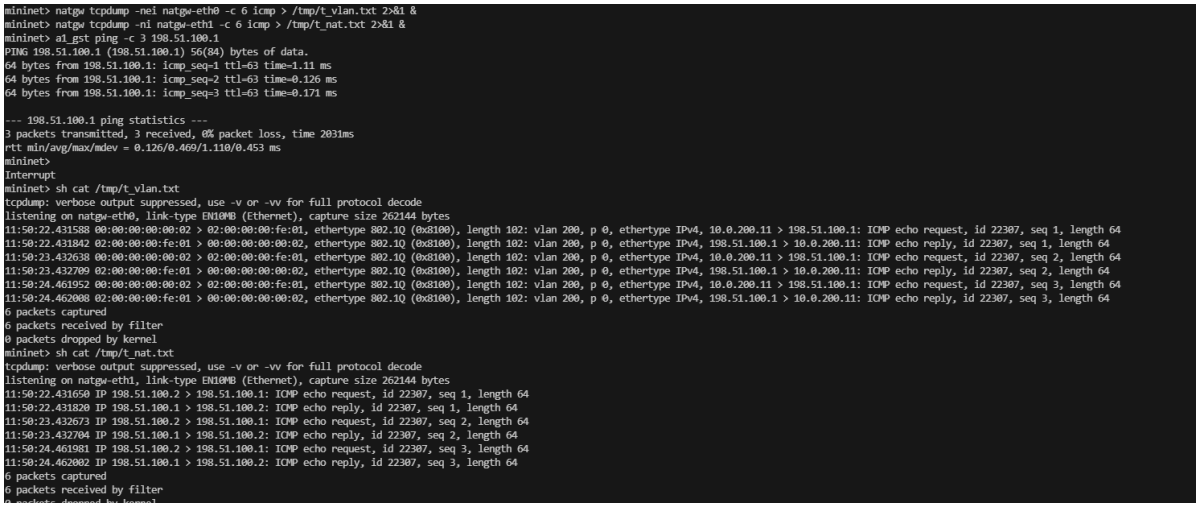
```
mininet> isp_a ip route add 10.0.100.0/23 via 198.51.100.2
mininet> isp_a traceroute 10.0.100.11
traceroute to 10.0.100.11 (10.0.100.11), 30 hops max, 60 byte packets
 1  198.51.100.2 (198.51.100.2)  0.418 ms  0.019 ms  0.012 ms
 2  * * *
 3  * * *
 4  * * *
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  * * *
10  * * *
11  * * *
12  * * *
13  * * *
14  * * *
15  * * *
16  * * *
17  * * *
18  * * *
19  * * *
20  * * *
21  * * *
22  * * *
23  * * *
24  * * *
25  * * *
26  * * *
27  * * *
28  * * *
29  * * *
30  * * *
mininet>
```

Hop 1 is natgw (198.51.100.2), dus het pakket komt wel bij de gateway. Daarna volgen alleen sterretjes: er komt geen antwoord. De firewallregel op natgw laat alleen NEW-verbindingen van binnen naar buiten door en de FORWARD-policy dropt de rest. Dat zijn de pakketten die bij de DROP-teller in figuur 11 en 12 worden geteld. Zo is zichtbaar dat de firewall het werkt: de gast bereikt de ISP (traceroute hierboven) maar de ISP bereikt het interne netwerk niet.

4.4.4 Pakketten op natgw (tcpdump)

Doel: aantonen dat het verkeer met een VLAN-tag op natgw aankomt en dat het adres naar buiten wordt vertaald (PAT).

```
mininet> natgw tcpdump -nei natgw-eth0 -c 6 icmp > /tmp/t_vlan.txt 2>&1 &
mininet> natgw tcpdump -ni natgw-eth1 -c 6 icmp > /tmp/t_nat.txt 2>&1 &
mininet> a1_gst ping -c 3 198.51.100.1
mininet> sh cat /tmp/t_vlan.txt
mininet> sh cat /tmp/t_nat.txt
```



Binnenkant (natgw-eth0). De frames hebben ethertype 802.1Q met vlan 200. Het bron-MAC is dat van a1_gst (00:00:00:00:00:02), het doel-MAC is het gateway-MAC (02:00:00:00:fe:01) en het IP-adres is 10.0.200.11. Dit bevestigt de trunk en de ACL-regel die op het gateway-MAC toelaat.

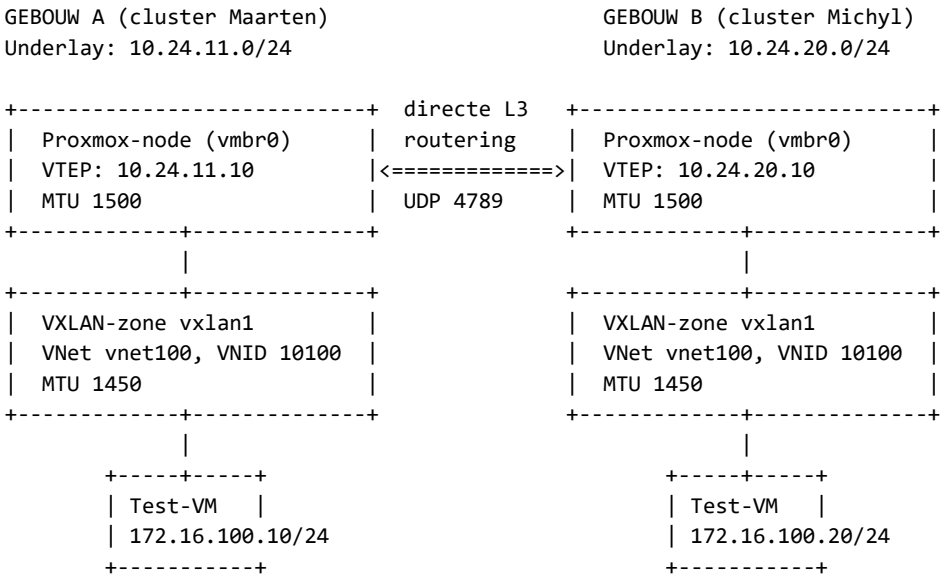
Buitenkant (natgw-eth1). Dezelfde pings (id 22307) komen zonder VLAN-tag en met bronadres 198.51.100.2 naar buiten. Dat is de SNAT-regel (PAT) van natgw. Het antwoord komt terug op 198.51.100.2 en natgw vertaalt het weer naar 10.0.200.11.

4.5 Opdracht 4: Overlay-netwerk met VxLAN

4.5.1 Context en doelstelling

Opdracht 4 heb ik samen met Maarten uitgevoerd. In deze opdracht bouwen we een overlay-netwerk over twee gescheiden Proxmox VE 9.2-clusters: het cluster van Maarten (Gebouw A, PVEVT02) en mijn cluster (Gebouw B, PVEVT03). Met VxLAN (RFC 7348) leggen we over de bestaande Laag 3-underlay één gedeeld Laag 2/Laag 3-subnet, 172.16.100.0/24. Een VM in het ene cluster en een VM in het andere cluster zitten daardoor in hetzelfde subnet, alsof ze aan dezelfde switch hangen. Dit sluit aan op de theorie over overlay-netwerken en de VTEP's in hoofdstuk 3 en op de eis om twee locaties transparant te koppelen.

4.5.2 Netwerktekening en architectuur



Onderdeel	Waarde
Underlay Gebouw A / B	10.24.11.0/24 / 10.24.20.0/24 (gerouteerd, via vmbr0)
VxLAN-poort	UDP 4789, niet geblokkeerd in het datacenter
SDN-zone / VNet	vxlan1 / vnet100
VNID	10100
Overlay-subnet	172.16.100.0/24 (gateway 172.16.100.1)
MTU overlay	1450 bytes (underlay 1500)
Test-VM's	172.16.100.10 (Gebouw A) en 172.16.100.20 (Gebouw B)

4.5.3 Onderbouwing ontwerpkeuzes

• Directe L3-underlay zonder WireGuard. Uit netwerktests bleek dat de Proxmox-nodes in 10.24.11.0/24 en 10.24.20.0/24 elkaar al via de standaardgateway konden bereiken en dat UDP-poort 4789 niet werd geblokkeerd. Onze eerste aanpak met handmatige WireGuard-tunnels werkte niet goed samen met de interfaces.d/sdn-stack van Proxmox en veroorzaakte routing loops op vmbr0. Een extra VPN-laag was dus niet nodig. Zonder WireGuard vervalt ook de encryptie-overhead en de kans op asymmetrische routering.

• VTEP-bepaling. De Proxmox-nodes zelf zijn de VxLAN Tunnel Endpoints. De tunnel wordt centraal beheerd via de Proxmox Datacenter SDN, waarbij de IP-adressen van de vmbr0-interfaces in de peers-lijst van de zone staan. Na het aanmaken moet de SDN-configuratie met Apply worden toegepast; pas dan opent de kernel poort 4789.

• VNID en subnet. We gebruiken VNID 10100 voor het private subnet 172.16.100.0/24. De nummering is logisch opgebouwd (10100 hoort bij 100.0/24) en het subnet overlapt niet met de lokale netwerken.

4.5.4 MTU-berekening

VxLAN pakt elk Ethernet-frame in een nieuw pakket. Dat kost extra headerbytes en moet binnen de MTU van de underlay passen, anders wordt het pakket gefragmenteerd.

• MTU van de underlay (vmbr0): 1500 bytes.

• Overhead van de encapsulatie: buiten-IPv4 (20) + UDP (8) + VxLAN-header (8) + binnen-Ethernet (14) = 50 bytes.

• MTU van de overlay (vnet100 en de VM-netwerkkaarten): 1500 − 50 = 1450 bytes.

4.5.5 Bewijs en tests

M1. Laag 3-underlay connectiviteit

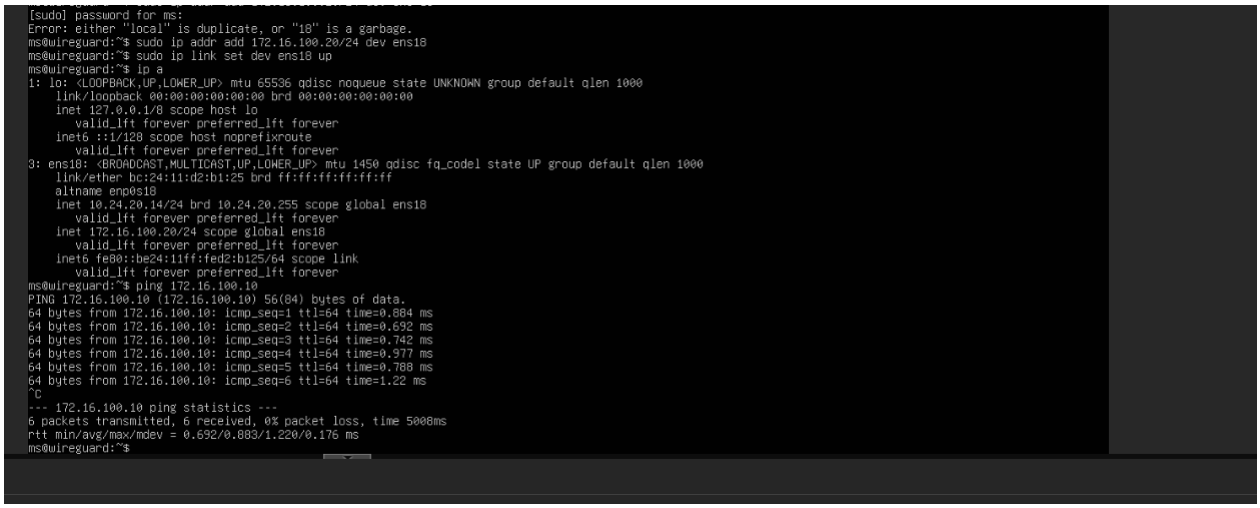
Doel: aantonen dat de VTEP-nodes elkaar over het fysieke netwerk kunnen bereiken.

```
root@vm1:~# ping -c 3 10.24.20.2
```

Resultaat: 0% packet loss. De gerouteerde underlay werkt.

```
root@vm1:~# ping -c 3 10.24.20.2
PING 10.24.20.2 (10.24.20.2) 56(84) bytes of data.
64 bytes from 10.24.20.2: icmp_seq=1 ttl=63 time=0.208 ms
64 bytes from 10.24.20.2: icmp_seq=2 ttl=63 time=0.248 ms
64 bytes from 10.24.20.2: icmp_seq=3 ttl=63 time=0.190 ms

--- 10.24.20.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2068ms
rtt min/avg/max/mdev = 0.190/0.215/0.248/0.024 ms
```

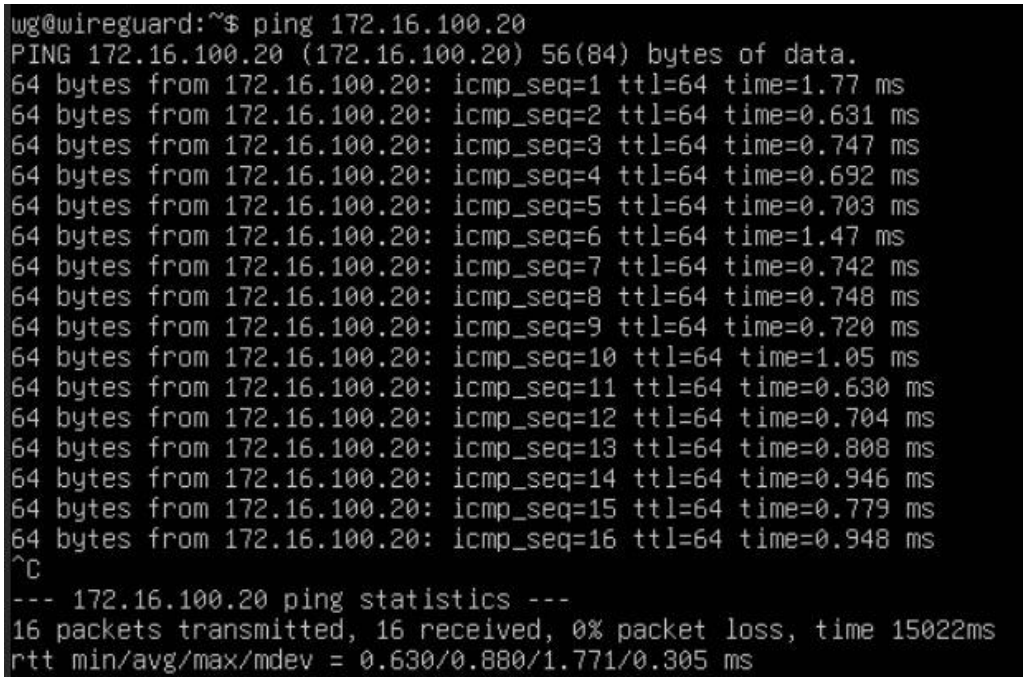


M2. Overlay-connectiviteit binnen één subnet

Doel: aantonen dat hosts in het overlay-netwerk elkaar kunnen pingen.

```
wg@wireguard:~$ ping 172.16.100.20
```

Resultaat: 0% packet loss. De hosts bereiken elkaar direct op Laag 2 via de VxLAN-tunnel.

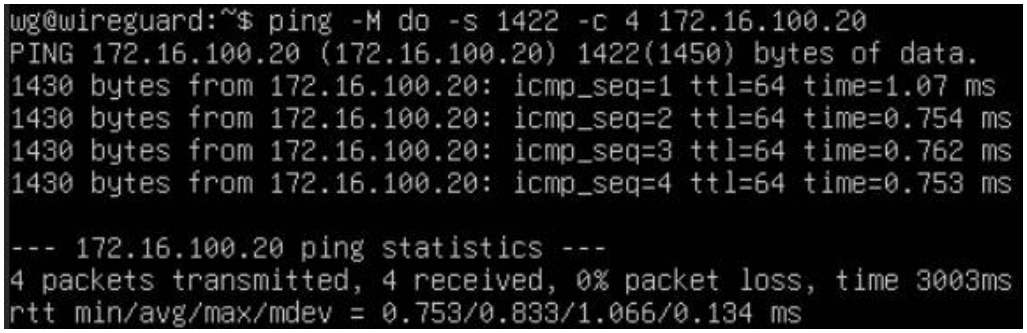


M3. MTU- en fragmentatietest

Doel: bewijzen dat de berekende MTU van 1450 klopt. We sturen een ping met het Don't Fragment-bit; 1450 − 28 bytes ICMP/IP-header = 1422 bytes payload.

```
wg@wireguard:~$ ping -M do -s 1422 -c 4 172.16.100.20
```

Resultaat: de pings slagen, dus de pakketten worden zonder fragmentatie ingekapseld.



5 .Reflectie en evaluatie

Zijn de doelstellingen bereikt?

De doelstellingen van opdracht 2 en 3 zijn grotendeels bereikt. Het netwerk met twee gebouwen werkt, alle zeven switches worden door Faucet beheerd en de VLAN's lopen over de darkfiber. Ook de NFV-functies werken: rate limiting voor gasten en port security op het managementnetwerk. Het netwerk draait met IPv4 en IPv6, en een stateful firewall met NAT beschermt de uitgang naar de ISP. De tests laten dit zien, met pings, iperf en tellers in de flow tables en in iptables.

Bij een paar onderdelen is de doelstelling maar gedeeltelijk gehaald. De test voor 500 gebruikers gebruikt één host met 500 adressen en dus één MAC-adres. Die test bewijst NAT en conntrack, maar niet dat Faucet 500 verschillende MAC-adressen leert. Ook heb ik de ISP-kant sterk vereenvoudigd.

Wat ging goed en wat ging lastig?

Het begin ging goed: de IPv4-opzet en de basis van het netwerk stonden relatief snel. De VLAN's, de trunks en de communicatie tussen de twee gebouwen werkten snel, zodat ik daarop kon verder bouwen.

Lastiger was de oplossing voor het /28-blok en de ISP. Ik heb lang gezocht naar de beste optie. Eén publiek /28 betekent één NAT-punt. Een uplink per gebouw zou dus een tweede publiek blok of een route tussen twee gateways vragen. Uiteindelijk koos ik voor één gedeelde uplink met PAT naar één publiek adres.

De Linux-gateway (natgw) leverde de meeste problemen op. Die doet de routering, NAT en de stateful firewall. Mijn eerste firewall blokkeerde employee naar management met stateless ACL's in Faucet, maar die dropten ook het terugverkeer van een verbinding. Daarom heb ik dat beleid naar natgw verplaatst, waar conntrack de richting van een verbinding kent. Een ander probleem was dat het MAC-adres van natgw niet klopte met wat de gast-ACL verwachtte, waardoor gasten de ISP niet bereikten. Dat kwam pas aan het licht toen ik de flows en tellers bekeek.

Was de aanpak effectief?

De aanpak was effectief, maar niet perfect. Eén Faucet-YAML met een vaste tabelstructuur maakt het netwerk reproduceerbaar. Door na elke wijziging te testen met pings en tellers vond ik fouten snel. Verbeterpunten zijn er ook. Ik heb het netwerk grotendeels met de hand opgebouwd, met veel herhaling in de YAML (per managementpoort een eigen ACL). Met sjablonen of een script was dat compacter en minder foutgevoelig geweest. Ook ging het testen sneller als ik het bewijs vanaf het begin had opgeslagen, in plaats van het later te verzamelen.

Wat zou je de volgende keer anders doen?

Ik zou eerder uitzoeken hoe Mininet het netwerk opzet, bijvoorbeeld hoe poortnummers en interfacenamen worden toegewezen. Ik zou ook eerder bepalen hoe ik de OpenFlow-tabellen het beste weergeef. In het begin had ik daar veel moeite mee om de juiste informatie te krijgen. Een

dump van een switch is lang en de tabelnummers verschillen per switch, omdat de core-switches geen ACL's hebben. Als ik van tevoren filters en een vaste werkwijze had bepaald, had dat veel tijd gescheeld.

Wat heb je geleerd en toegepast?

Ik heb geleerd hoe lastig het is om een groot netwerk in een topologie neer te zetten. Het is makkelijk om een typefout te maken waardoor de syntax niet werkt, of om per ongeluk een 4 in plaats van een 3 voor een switch te zetten. Een klein verschil tussen de Mininet-topologie en de Faucet-YAML, zoals een verkeerd datapath-ID, zorgt er direct voor dat een switch niet verbindt. Voor de regels in de ACL's, de firewall en de flow tables was het ook goed zoeken naar wat precies werkte. De volgorde en prioriteit van regels bepalen het gedrag, en een regel op de verkeerde plek kan alles blokkeren.

Daarnaast heb ik de theorie toegepast: SDN met OpenFlow 1.3, VLAN's en trunking, router-on-a-stick, NAT/PAT, IPv6, meters en stateful filtering met conntrack. Door het zelf te bouwen en de tellers te bekijken, begrijp ik nu hoe een pakket door de tabellen loopt.

Welke uitbreiding zou je met meer tijd of middelen doen?

Met meer tijd zou ik de firewall en IPv6 verder uitwerken. Daar ben ik zelf nog niet het sterkst in en ik moest veel onderzoek doen om te kijken wat het beste werkte met Faucet en Mininet. Dat zou ik verdiepen, bijvoorbeeld met SLAAC of DHCPv6 in plaats van statische adressen, en met strengere regels op INPUT van natgw.

Verder zou ik natgw redundant maken, bijvoorbeeld met VRRP, omdat hij nu een single point of failure is. Ook zou ik de controller redundant maken en een tweede uplink voor gebouw B toevoegen. Voor port security zou ik naar max_hosts of 802.1X kijken. Tot slot zou ik de test voor 500 gebruikers uitbreiden met aparte MAC-adressen, zodat ook het leren in Faucet op schaal wordt aangetoond.

Reflectie opdracht 4: bouwen van een overlay-netwerk

Tijdens de VxLAN-overlay hebben we praktijkervaring opgedaan met Proxmox VE 9.2 SDN. In het begin liepen we vast: we gebruikten handmatige WireGuard-configuraties die niet goed samenwerkten met de interfaces.d/sdn-stack en routing loops veroorzaakten. Toen we goed naar de eisen van het netwerk keken, kwamen we verder. De underlay (10.24.11.0/24 en 10.24.20.0/24) kon elkaar al pingen en UDP 4789 stond open, dus de WireGuard- en SDN Fabric-laag was overbodig. Door te kiezen voor een directe L3-underlay legden we de VxLAN-zone direct over de bestaande vmbr0-interfaces. Dat hield de overhead bij 50 bytes (MTU 1450) en gaf een werkende overlay. Wat ik ervan leerde: eerst controleren wat de bestaande infrastructuur al kan, voordat je extra lagen toevoegt

6.Literatuurlijst

1. Mininet Project. *Mininet: Rapid Prototyping for Software Defined Networks* (README). GitHub / Software Heritage. 2026 (archief). https://archive.softwareheritage.org/browse/content/sha1_git:4c668ae85682a8c98a28b3124c3ce2003a79e478/

   https://www.youtube.com/watch?v=Glb88drqrOg

2. Faucet SDN Team. *FAUCET: OpenFlow controller for multi-table OpenFlow 1.3 switches*. GitHub. 2026. https://github.com/faucetsdn/faucet

3. Open Networking Foundation. *OpenFlow Switch Specification 1.3.4*. Open Networking Foundation. 2014. https://opennetworking.org/sdn-resources/openflow-switch-specification/

4. Mahalingam, M. et al. *RFC 7348: Virtual eXtensible Local Area Network (VXLAN)*. IETF Datatracker. 2014. https://datatracker.ietf.org/doc/rfc7348/

5. Arcfra. *Data plane* (ANS distributed firewall technical whitepaper). Arcfra Docs. 2026. https://docs.arcfra.com/deep-dive/latest/ans/network-service/3.5.0/ans_whitepaper/whitepaper_09

6. ETSI NFV ISG (via IETF I2NSF). *Network Security Function (NSF) definitie* (in *I2NSF Gap Analysis*). IETF Datatracker. 2016. https://datatracker.ietf.org/meeting/96/agenda/i2nsf-drafts.pdf

7. IETF I2NSF Working Group. *Applicability of I2NSF to Network Security Services* (presentatie). IETF Datatracker. 2018. https://datatracker.ietf.org/meeting/101/materials/slides-101-i2nsf-applicability-to-network-security-services-02

8. Proxmox Server Solutions. Proxmox VE Administration Guide: Software-Defined Network. pve.proxmox.com. 2026. https://pve.proxmox.com/pve-docs/chapter-pvesdn.html

9. "Anthropic. Claude (AI-assistent). 2026. https://claude.ai".

Bijlage 1 – Logboek

Datum	Activiteit	Beschrijving / Resultaat	Reflectie / geleerde
07/09/2026	Opzet portfolio en PVA	Tot sectie 3 gekomen	Zorgvuldig door de opdrachten lezen
09/09/2026	PVA verder afmaken	PVA afgemaakt	Hoe de omgeving werkt met alle verschillende applicaties
17/09/2026	VM's opzetten	Juiste instellingen voor de VM's gekozen	Faucet en Mininet draaien op aparte VM's; de switches verbinden met de controller via OpenFlow op poort 6653
28/09/2026	Research gedaan op ip subnets	Subnetindeling gekozen: employee /23 (ruimte voor 500 gebruikers), gast /24, management /24 en één publiek /28	Meer insight op subnetten
30/09/2026	Research voor rate limiter	Gekozen voor een OpenFlow-meter in Faucet in plaats van tc op de hosts	Juiste tool voor rate limiting gevonden
01/10/2026	Eerste deel van de topologie en Faucet-YAML gemaakt	Groot deel van de basis neergezet	Syntax van de modules geleerd
03/10/2026	Rate limiting en port security maken	Deels af gemaakt	Hoe port security in Faucet werkt en hoe de rate limiter werkt
04/10/2026	Rate limiting afmaken	Rate limiting werkt en is testbaar	Goed moeten kijken welke flags nodig waren voor rate limiting
06/10/2026	Onderzoeken hoe IPv6 toegevoegd kan worden	Adresplan gemaakt: een eigen /64 per VLAN (2001:db8:100, 200 en 300) en 2001:db8:ffff::/64 naar de ISP	Gekeken naar IPv6-adressering en hoe Mininet daarmee omgaat
07/10/2026	IPv6 neerzetten	IPv6-adressen en default routes op hosts en natgw ingesteld	IPv6 werkt pas als adressen, gateway en terugroutes allemaal kloppen

11/10/2026	Vastgelopen op IPv6	IPv6-adressen kwamen niet op de interfaces en het verkeer werkte niet	Kon adressen niet laden
15/10/2026	Kernel-instellingen voor IPv6 aanpassen	IPv6-forwarding op natgw aangezet	Routering moet op Linux per protocol (IPv4 en IPv6) worden ingeschakeld
18/10/2026	Firewall regels toepassen	Stateful firewall met conntrack op natgw voor IPv4 en IPv6: INVALID droppen, ESTABLISHED en RELATED toestaan, NEW alleen van binnen naar buiten	Een stateless Faucet-ACL kan retourverkeer niet herkennen, daarom staat dit beleid op natgw
21/10/2026	Samen met Maarten aan de underlay voor VxLAN gewerkt	Vastgelopen op de VPN-tunnel	Geleerd hoe de VPN werkt, maar het was niet de juiste oplossing
26/10/2026	Met medestudenten van een andere groep gesproken	Uitleg gekregen over het instellen van SDN in Proxmox	Geleerd hoe je in Proxmox een SDN-instelling aanpast
26/10/2026	Directe L3-underlay gekozen en VxLAN-zone ingesteld	Zone vxlan1 en vnet100 (VNID 10100) in Proxmox Datacenter geconfigureerd, peers = vmbr0-adressen van beide clusters	SDN-configuratie moet met Apply worden toegepast, anders opent poort 4789 niet
27/10/2026	Opdracht 4 afgerond	MTU berekend op 1450; ping, ping -M do -s 1422 en underlay-ping geslaagd	Eerst kijken wat de bestaande underlay al kan voordat je een VPN toevoegt

Bijlage 2 – Code

Bijlage 2.1 – Faucet YAML

Faucet-configuratie van alle zeven switches (VLAN's, meter, ACL's en poorten). Bestand Faucet.yml in de repository: https://github.com/MichylS/SDN

```
---
# Faucet-configuratie voor drie gescheiden netwerken:
# employee (kantoor), guest (bezoekers) en management.
# De VLAN's worden door de core- en verdiepingsswitches getagd
# doorgestuurd naar de router/gateway.
meters:
    # Guest-verkeer wordt per switchpoort beperkt tot 1 Mbit/s.
    guest-rate-limit:
        meter_id: 1
        entry:
            flags: [KBPS, BURST, STATS]
            bands:
                - type: DROP
                  rate: 1000
                  burst_size: 100


acls:
    # ACL's worden toegepast op inkomend verkeer van eindhosts.
    # Employee mag op switchniveau niet rechtstreeks naar guest.
    # Overige toegangsbeperkingen worden op de gateway afgehandeld.
    office-acl:
        - rule:
              description: "Employee -> guest verboden (IPv4)"
              dl_type: 0x800
              ipv4_dst: "10.0.200.0/24"
              actions:
                  allow: 0
        - rule:
              description: "Employee -> guest verboden (IPv6)"
              dl_type: 0x86dd
              ipv6_dst: "2001:db8:200::/64"
              actions:
                  allow: 0
        - rule:
              description: "Rest toegestaan"
              actions:
                  allow: 1


    # Guest mag alleen via de gateway naar buiten en krijgt 1 Mbit/s
    # per poort. Verkeer naar interne netwerken wordt expliciet geweigerd.
    guest-acl:
        - rule:
              description: "ARP toestaan"
              dl_type: 0x806
              actions:
                  allow: 1
        - rule:
              description: "IPv6 neighbour solicitation"
              dl_type: 0x86dd
              ip_proto: 58
              icmpv6_type: 135
              actions:
                  allow: 1
        - rule:
              description: "IPv6 neighbour advertisement"
              dl_type: 0x86dd
              ip_proto: 58
              icmpv6_type: 136
              actions:
                  allow: 1
        - rule:
              description: "Guest -> eigen gateway IPv4"
```

```
        dl_type: 0x800
        eth_dst: "02:00:00:00:fe:01"
        ipv4_dst: "10.0.200.254/32"
        actions:
            meter: guest-rate-limit
            allow: 1
    - rule:
        description: "Guest -> guest verboden (IPv4)"
        dl_type: 0x800
        ipv4_dst: "10.0.200.0/24"
        actions:
            allow: 0
    - rule:
        description: "Guest -> employee verboden (IPv4)"
        dl_type: 0x800
        ipv4_dst: "10.0.100.0/23"
        actions:
            allow: 0
    - rule:
        description: "Guest -> management verboden (IPv4)"
        dl_type: 0x800
        ipv4_dst: "10.0.30.0/24"
        actions:
            allow: 0
    - rule:
        description: "Guest -> ISP via gateway (IPv4), rate limited"
        dl_type: 0x800
        eth_dst: "02:00:00:00:fe:01"
        actions:
            meter: guest-rate-limit
            allow: 1
    - rule:
        description: "Guest -> eigen gateway IPv6"
        dl_type: 0x86dd
        eth_dst: "02:00:00:00:fe:01"
        ipv6_dst: "2001:db8:200::1/128"
        actions:
            meter: guest-rate-limit
            allow: 1
    - rule:
        description: "Guest -> guest verboden (IPv6)"
        dl_type: 0x86dd
        ipv6_dst: "2001:db8:200::/64"
        actions:
            allow: 0
    - rule:
        description: "Guest -> employee verboden (IPv6)"
        dl_type: 0x86dd
        ipv6_dst: "2001:db8:100::/64"
        actions:
            allow: 0
    - rule:
        description: "Guest -> management verboden (IPv6)"
        dl_type: 0x86dd
        ipv6_dst: "2001:db8:300::/64"
        actions:
            allow: 0
    - rule:
        description: "Guest -> ISP via gateway (IPv6), rate limited"
        dl_type: 0x86dd
        eth_dst: "02:00:00:00:fe:01"
        actions:
            meter: guest-rate-limit
            allow: 1
    - rule:
        description: "Al het andere droppen"
        actions:
            allow: 0

# Ethernet-port security voor management:
# elke managementpoort accepteert slechts één vaste bron-MAC.
port-security-a1-mgt:
    - rule:
```

```
            description: "Alleen a1_mgt"
            eth_src: "02:00:00:00:30:11"
            actions:
                allow: 1
        - rule:
            description: "Overige MAC-adressen droppen"
            actions:
                allow: 0
    port-security-a2-mgt:
        - rule:
            description: "Alleen a2_mgt"
            eth_src: "02:00:00:00:30:12"
            actions:
                allow: 1
        - rule:
            description: "Overige MAC-adressen droppen"
            actions:
                allow: 0
    port-security-b1-mgt:
        - rule:
            description: "Alleen b1_mgt"
            eth_src: "02:00:00:00:30:21"
            actions:
                allow: 1
        - rule:
            description: "Overige MAC-adressen droppen"
            actions:
                allow: 0
    port-security-b2-mgt:
        - rule:
            description: "Alleen b2_mgt"
            eth_src: "02:00:00:00:30:22"
            actions:
                allow: 1
        - rule:
            description: "Overige MAC-adressen droppen"
            actions:
                allow: 0
    port-security-b3-mgt:
        - rule:
            description: "Alleen b3_mgt"
            eth_src: "02:00:00:00:30:23"
            actions:
                allow: 1
        - rule:
            description: "Overige MAC-adressen droppen"
            actions:
                allow: 0


vlans:
    # Native VLAN's worden gebruikt op accesspoorten; op trunks zijn
    # dezelfde VLAN's als tagged VLAN toegestaan.
    office:
        vid: 100
        description: "Employee network 10.0.100.0/23 en 2001:db8:100::/64"
    guest:
        vid: 200
        description: "Guest network 10.0.200.0/24 en 2001:db8:200::/64"
    management:
        vid: 300
        description: "Management network 10.0.30.0/24 en 2001:db8:300::/64"


dps:
    # Elke datapath switch (DP) gebruikt een uniek datapath-ID.
    # Core-switches verbinden verdiepingen en de gateway; verdiepings-
    # switches verbinden de eindhosts met accesspoorten.
    a_core:
        dp_id: 0x1
        hardware: "Open vSwitch"
        interfaces:
            1:
                description: "Trunk naar a_f1"
```

```
            tagged_vlans: [office, guest, management]
        2:
            description: "Trunk naar a_f2"
            tagged_vlans: [office, guest, management]
        3:
            description: "Darkfiber naar b_core"
            tagged_vlans: [office, guest, management]
        4:
            description: "Trunk naar natgw (router, NAT en stateful firewall)"
            tagged_vlans: [office, guest, management]

a_f1:
    dp_id: 0x2
    hardware: "Open vSwitch"
    interfaces:
        1:
            description: "Trunk naar a_core"
            tagged_vlans: [office, guest, management]
        2:
            description: "Employee a1_emp"
            native_vlan: office
            acls_in: [office-acl]
        3:
            description: "Guest a1_gst"
            native_vlan: guest
            acls_in: [guest-acl]
        4:
            description: "Management a1_mgt"
            native_vlan: management
            acls_in: [port-security-a1-mgt]

a_f2:
    dp_id: 0x3
    hardware: "Open vSwitch"
    interfaces:
        1:
            description: "Trunk naar a_core"
            tagged_vlans: [office, guest, management]
        2:
            description: "Employee a2_emp"
            native_vlan: office
            acls_in: [office-acl]
        3:
            description: "Guest a2_gst"
            native_vlan: guest
            acls_in: [guest-acl]
        4:
            description: "Management a2_mgt"
            native_vlan: management
            acls_in: [port-security-a2-mgt]

b_core:
    dp_id: 0x4
    hardware: "Open vSwitch"
    interfaces:
        1:
            description: "Trunk naar b_f1"
            tagged_vlans: [office, guest, management]
        2:
            description: "Trunk naar b_f2"
            tagged_vlans: [office, guest, management]
        3:
            description: "Trunk naar b_f3"
            tagged_vlans: [office, guest, management]
        4:
            description: "Darkfiber naar a_core"
            tagged_vlans: [office, guest, management]

b_f1:
    dp_id: 0x5
    hardware: "Open vSwitch"
```

```
    interfaces:
        1:
            description: "Trunk naar b_core"
            tagged_vlans: [office, guest, management]
        2:
            description: "Employee b1_emp"
            native_vlan: office
            acls_in: [office-acl]
        3:
            description: "Guest b1_gst"
            native_vlan: guest
            acls_in: [guest-acl]
        4:
            description: "Management b1_mgt"
            native_vlan: management
            acls_in: [port-security-b1-mgt]

b_f2:
    dp_id: 0x6
    hardware: "Open vSwitch"
    interfaces:
        1:
            description: "Trunk naar b_core"
            tagged_vlans: [office, guest, management]
        2:
            description: "Employee b2_emp"
            native_vlan: office
            acls_in: [office-acl]
        3:
            description: "Guest b2_gst"
            native_vlan: guest
            acls_in: [guest-acl]
        4:
            description: "Management b2_mgt"
            native_vlan: management
            acls_in: [port-security-b2-mgt]

b_f3:
    dp_id: 0x7
    hardware: "Open vSwitch"
    interfaces:
        1:
            description: "Trunk naar b_core"
            tagged_vlans: [office, guest, management]
        2:
            description: "Employee b3_emp"
            native_vlan: office
            acls_in: [office-acl]
        3:
            description: "Guest b3_gst"
            native_vlan: guest
            acls_in: [guest-acl]
        4:
            description: "Management b3_mgt"
            native_vlan: management
            acls_in: [port-security-b3-mgt]
```

Bijlage 2.2 – Mininet Python

Mininet-topologie met de zeven Open vSwitch-switches, de hosts, natgw en de ISP-segment.

```
#!/usr/bin/env python3
import argparse
from mininet.cli import CLI
from mininet.link import TCLink
from mininet.log import info, setLogLevel
from mininet.net import Mininet
from mininet.node import OVSBridge, OVSKernelSwitch, RemoteController
from mininet.topo import Topo
from mininet.util import quietRun


# MAC van de binnenkant van natgw. Faucet-ACL's matchen hierop.
NATGW_MAC = "02:00:00:00:fe:01"
INSIDE = "natgw-eth0"
OUTSIDE = "natgw-eth1"


VLANS = {
    "emp": {"vid": 100, "net4": "10.0.100", "plen4": 23, "net6": "2001:db8:100"},
    "gst": {"vid": 200, "net4": "10.0.200", "plen4": 24, "net6": "2001:db8:200"},
    "mgt": {"vid": 300, "net4": "10.0.30", "plen4": 24, "net6": "2001:db8:300"},
}


# (prefix, verdiepingsswitch, hostnummer)
FLOORS = [
    ("a1", "a_f1", 11),
    ("a2", "a_f2", 12),
    ("b1", "b_f1", 21),
    ("b2", "b_f2", 22),
    ("b3", "b_f3", 23),
]


# Poortindeling op elke verdiepingsswitch.
PORTS = {"emp": 2, "gst": 3, "mgt": 4}


# (naam, dpid)
SWITCHES = [
    ("a_core", 1),
    ("a_f1", 2),
    ("a_f2", 3),
    ("b_core", 4),
    ("b_f1", 5),
    ("b_f2", 6),
    ("b_f3", 7),
]



def gw4(kind):
    return "%s.254" % VLANS[kind]["net4"]



def gw6(kind):
    return "%s::1" % VLANS[kind]["net6"]



class BuildingsTopo(Topo):
    def build(self):
        for name, dpid in SWITCHES:
            self.addSwitch(
                name,
                dpid="%016x" % dpid,
                protocols="OpenFlow13",
                failMode="secure",
            )
```

```
        # ISP-kant: gewone bridge, geen Faucet.
        self.addSwitch("isp_net", cls=OVSBridge, dpid="%016x" % 8)


        # Gateway en ISP-hosts. natgw krijgt zijn adressen later.
        self.addHost("natgw", ip=None, mac=NATGW_MAC)
        self.addHost("isp_a", ip="198.51.100.1/28")
        self.addHost("isp_b", ip="198.51.100.3/28")


        # Hosts per verdieping.
        for prefix, switch, number in FLOORS:
            for kind, vlan in VLANS.items():
                name = "%s_%s" % (prefix, kind)
                params = {"ip": "%s.%d/%d" % (vlan["net4"], number, vlan["plen4"])}
                link_opts = {"port1": PORTS[kind], "port2": 0}
                if kind == "mgt":
                    mac = "02:00:00:00:30:%d" % number
                    params["mac"] = mac
                    link_opts["addr2"] = mac
                self.addHost(name, **params)
                self.addLink(switch, name, **link_opts)

        # Gebouw A: core naar verdiepingen.
        self.addLink("a_core", "a_f1", port1=1, port2=1)
        self.addLink("a_core", "a_f2", port1=2, port2=1)


        # Gebouw B: core naar verdiepingen.
        self.addLink("b_core", "b_f1", port1=1, port2=1)
        self.addLink("b_core", "b_f2", port1=2, port2=1)
        self.addLink("b_core", "b_f3", port1=3, port2=1)


        # Darkfiber tussen de cores.
        self.addLink("a_core", "b_core", port1=3, port2=4)


        # Gateway op a_core poort 4 (trunk met alle VLAN's).
        self.addLink("a_core", "natgw", port1=4, port2=0, addr2=NATGW_MAC)


        # Publiek /28-segment.
        self.addLink("natgw", "isp_net", port1=1)
        self.addLink("isp_net", "isp_a")
        self.addLink("isp_net", "isp_b")



def enable_ipv6(node, interface):
    node.cmd("sysctl -w net.ipv6.conf.all.disable_ipv6=0")
    node.cmd("sysctl -w net.ipv6.conf.default.disable_ipv6=0")
    node.cmd("sysctl -w net.ipv6.conf.%s.disable_ipv6=0" % interface)



def configure_natgw(net):
    gw = net["natgw"]
    quietRun("modprobe 8021q")


    for command in (
        "sysctl -w net.ipv4.ip_forward=1",
        "sysctl -w net.ipv6.conf.all.disable_ipv6=0",
        "sysctl -w net.ipv6.conf.default.disable_ipv6=0",
        "sysctl -w net.ipv6.conf.all.forwarding=1",
        "sysctl -w net.ipv4.conf.all.send_redirects=0",
        "sysctl -w net.ipv6.conf.%s.disable_ipv6=0" % INSIDE,
        "sysctl -w net.ipv6.conf.%s.disable_ipv6=0" % OUTSIDE,
        "ip addr flush dev %s" % INSIDE,
        "ip link set %s up" % INSIDE,
        "ip link set %s up" % OUTSIDE,
    ):
        gw.cmd(command)
```

```
    # Een subinterface per VLAN, met het gateway-adres van dat VLAN.
    for kind, vlan in VLANS.items():
        sub = "%s.%d" % (INSIDE, vlan["vid"])
        gw.cmd("ip link add link %s name %s type vlan id %d" % (INSIDE, sub, vlan["vid"]))
        gw.cmd("ip link set %s up" % sub)
        gw.cmd("ip addr add %s/%d dev %s" % (gw4(kind), vlan["plen4"], sub))
        gw.cmd("ip -6 addr add %s/64 dev %s nodad" % (gw6(kind), sub))


    # Publieke kant.
    gw.cmd("ip addr add 198.51.100.2/28 dev %s" % OUTSIDE)
    gw.cmd("ip -6 addr add 2001:db8:ffff::2/64 dev %s nodad" % OUTSIDE)
    gw.cmd("ip route replace default via 198.51.100.1")
    gw.cmd("ip -6 route replace default via 2001:db8:ffff::1 dev %s" % OUTSIDE)


    emp = "%s.%d" % (INSIDE, VLANS["emp"]["vid"])
    gst = "%s.%d" % (INSIDE, VLANS["gst"]["vid"])
    mgt = "%s.%d" % (INSIDE, VLANS["mgt"]["vid"])


    for tool in ("iptables", "ip6tables"):
        gw.cmd("%s -F" % tool)
        gw.cmd("%s -X" % tool)
        gw.cmd("%s -P INPUT DROP" % tool)
        gw.cmd("%s -P FORWARD DROP" % tool)
        gw.cmd("%s -P OUTPUT ACCEPT" % tool)
        gw.cmd("%s -A INPUT -i lo -j ACCEPT" % tool)
        gw.cmd("%s -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT" % tool)
        gw.cmd("%s -A INPUT -i %s.+ -j ACCEPT" % (tool, INSIDE))
        gw.cmd("%s -A FORWARD -m conntrack --ctstate INVALID -j DROP" % tool)
        gw.cmd("%s -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT" % tool)
        # Nieuwe verbindingen van binnen naar de ISP.
        for sub in (emp, gst, mgt):
            gw.cmd(
                "%s -A FORWARD -i %s -o %s -m conntrack --ctstate NEW -j ACCEPT"
                % (tool, sub, OUTSIDE)
            )
        # Management mag verbindingen starten naar employee en guest.
        for target in (emp,):
            gw.cmd(
                "%s -A FORWARD -i %s -o %s -m conntrack --ctstate NEW -j ACCEPT"
                % (tool, mgt, target)
            )
        # Alles anders valt onder de FORWARD DROP-policy:
        # employee -> guest/management, guest -> guest (hairpin),
        # guest -> intern en alle nieuwe verbindingen vanaf de ISP.


    # IPv6 neighbour discovery op de publieke kant.
    for icmp_type in ("neighbour-solicitation", "neighbour-advertisement"):
        gw.cmd(
            "ip6tables -A INPUT -i %s -p ipv6-icmp --icmpv6-type %s -j ACCEPT"
            % (OUTSIDE, icmp_type)
        )


    # IPv4 PAT: alle interne adressen delen 198.51.100.2.
    gw.cmd("iptables -t nat -F")
    gw.cmd(
        "iptables -t nat -A POSTROUTING -s 10.0.0.0/8 -o %s "
        "-j SNAT --to-source 198.51.100.2" % OUTSIDE
    )



def configure_hosts(net):
    for prefix, _switch, number in FLOORS:
        for kind, vlan in VLANS.items():
            name = "%s_%s" % (prefix, kind)
            host = net[name]
            interface = "%s-eth0" % name
            enable_ipv6(host, interface)
            host.cmd("ip route replace default via %s" % gw4(kind))
```

```
            host.cmd(
                "ip -6 addr replace %s::%d/64 dev %s nodad"
                % (vlan["net6"], number, interface)
            )
            host.cmd("ip -6 route replace default via %s dev %s" % (gw6(kind), interface))



def configure_isp(net):
    for name, suffix in (("isp_a", "1"), ("isp_b", "3")):
        isp = net[name]
        interface = "%s-eth0" % name
        enable_ipv6(isp, interface)
        isp.cmd("ip -6 addr replace 2001:db8:ffff::%s/64 dev %s nodad" % (suffix, interface))
        # Terugroutes naar de interne IPv6-prefixen (geen NAT bij IPv6).
        for vlan in VLANS.values():
            isp.cmd(
                "ip -6 route replace %s::/64 via 2001:db8:ffff::2 dev %s"
                % (vlan["net6"], interface)
            )



def run_topology(controller_ip):
    net = Mininet(
        topo=BuildingsTopo(),
        controller=None,
        switch=OVSKernelSwitch,
        link=TCLink,
        autoSetMacs=True,
        autoStaticArp=False,
        autoPinCpus=False,
    )
    net.addController(RemoteController("faucet", ip=controller_ip, port=6653))

    info("*** Starting network\n")
    net.start()

    info("*** Waiting for switches to connect to Faucet\n")
    quietRun("sleep 6")

    info("*** Configuring natgw, hosts and ISP\n")
    configure_natgw(net)
    configure_hosts(net)
    configure_isp(net)

    info("*** Network ready\n")
    CLI(net)

    info("*** Stopping network\n")
    net.stop()



if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--controller-ip", required=True, help="IP address of the Faucet VM")
    args = parser.parse_args()

    setLogLevel("info")
    run_topology(args.controller_ip)
```

```
#!/usr/bin/env python3
import argparse
from mininet.cli import CLI
from mininet.link import TCLink
from mininet.log import info, setLogLevel
from mininet.net import Mininet
from mininet.node import OVSBridge, OVSKernelSwitch, RemoteController
from mininet.topo import Topo
from mininet.util import quietRun

# MAC van de binnenkant van natgw. Faucet-ACL's matchen hierop.
NATGW_MAC = "02:00:00:00:fe:01"
INSIDE = "natgw-eth0"
OUTSIDE = "natgw-eth1"

VLANS = {
   "emp": {"vid": 100, "net4": "10.0.100", "plen4": 23, "net6": "2001:db8:100"},
   "gst": {"vid": 200, "net4": "10.0.200", "plen4": 24, "net6": "2001:db8:200"},
   "mgt": {"vid": 300, "net4": "10.0.30", "plen4": 24, "net6": "2001:db8:300"},
}

# (prefix, verdiepingsswitch, hostnummer)
FLOORS = [
   ("a1", "a_f1", 11),
   ("a2", "a_f2", 12),
   ("b1", "b_f1", 21),
   ("b2", "b_f2", 22),
   ("b3", "b_f3", 23),
```

Bijlage 2.3 – Overige code/configuratie

Testscripts. De schaaltest (figuur: 500 gebruikers)

scale_test.sh

```
#!/bin/bash
# Gebruik: scale_test.sh [aantal] [interface] [doel] [seconden]
COUNT=${1:-500}
IFACE=${2:-a1_emp-eth0}
TARGET=${3:-198.51.100.1}
SECONDS_RUN=${4:-20}
DIR=/tmp/scale
rm -rf "$DIR"; mkdir -p "$DIR"


python3 -c '
import ipaddress, sys
count = int(sys.argv[1])
skip = {"10.0.100.11", "10.0.100.12", "10.0.100.21",
        "10.0.100.22", "10.0.100.23", "10.0.100.254"}
net = ipaddress.ip_network("10.0.100.0/23")
ips = [str(h) for h in net.hosts() if str(h) not in skip][:count]
print("\n".join(ips))
' "$COUNT" > "$DIR/ips.txt"


while read -r ip; do
    ip addr add "$ip/23" dev "$IFACE"
done < "$DIR/ips.txt"

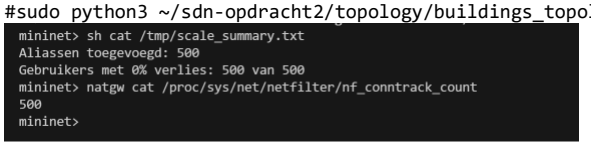

echo "Aliassen toegevoegd: $(wc -l < "$DIR/ips.txt")"
while read -r ip; do
    ping -c "$SECONDS_RUN" -i 1 -q -I "$ip" "$TARGET" > "$DIR/ping_$ip.txt" 2>&1 &
done < "$DIR/ips.txt"
wait


OK=$(grep -l ' 0% packet loss' "$DIR"/ping_*.txt | wc -l)
echo "Gebruikers met 0% verlies: $OK van $COUNT"


while read -r ip; do
    ip addr del "$ip/23" dev "$IFACE"
done < "$DIR/ips.txt"
```

Uitvoeren en meten (in de Mininet-CLI):

```
# om uit te voeren
# a1_emp bash /home/mininet/sdn-opdracht2/tests/scale_test.sh 500 a1_emp-eth0 198.51.100.1 20 >
/tmp/scale_summary.txt 2>&1 &
#meten
#natgw iptables -t nat -L POSTROUTING -n -v
#sh cat /tmp/scale_summary.txt
#sudo python3 ~/sdn-opdracht2/topology/buildings_topology.py --controller-ip 10.24.20.8
```



show_flows.sh

```
#!/bin/bash
# show_flows.sh: toont per figuur de flow entries uit hoofdstuk "b. Flow entries".
# Elk scherm begint met een banner met figuurnummer en sectie. Maak daarna je screenshot.
#
#   bash show_flows.sh          alle figuren, Enter = volgende, q = stoppen
#   bash show_flows.sh 2 6      alleen figuur 2 en 6
#   bash show_flows.sh lijst    overzicht: figuur -> sectie
#   FULL=1 bash show_flows.sh   volledige regels (met cookie, duration en timeouts)

SW_LIST="a_core a_f1 a_f2 b_core b_f1 b_f2 b_f3"

lijst() {
cat <<'TABEL'
Figuur  Sectie                                          Inhoud
------  ----------------------------------------------  ----------------------------------------
0       Overzicht                                       aantal flow entries per tabel per switch
1       Sectie 1: VLAN-segmentatie en trunking          a_f1 tabel 1
2       Sectie 4: Gastisolatie                          a_f1 tabel 0, gastpoort (poort 3)
3       Sectie 5: Rate limiting                         meter 1 en de flows die hem gebruiken
4       Sectie 5: Rate limiting                         meter-statistieken
5       Sectie 6: Port security                         a_f1 tabel 0, managementpoort (poort 4)
6       Sectie 2: MAC-learning en forwarding            a_f1 tabel 2 (leren)
7       Sectie 2: MAC-learning en forwarding            a_f1 tabel 3 (doorsturen)
8       Sectie 3: Flooding, ARP en IPv6 ND              a_f1 tabel 4 (flooding)
9       Sectie 7: Verkeer tussen verdiepingen           a_core tabel 0 en 2
10      Sectie 7: Darkfiber                             b_core tabel 0
11      Sectie 8: Stateful firewall IPv4                natgw iptables FORWARD
12      Sectie 8: Stateful firewall IPv6                natgw ip6tables FORWARD
13      Sectie 8: NAT                                   natgw iptables nat POSTROUTING
14      Sectie 9: Pad van een pakket (a_f1)             gast naar ISP, en gast naar gast
15      Sectie 9: Pad van een pakket (a_core)           gast naar ISP, heen en terug
TABEL
}

case "$1" in
  lijst|list|-h|--help) lijst; exit 0 ;;
esac

if ! sudo ovs-vsctl br-exists a_f1 2>/dev/null; then
  echo "Mininet draait niet (bridge a_f1 niet gevonden). Start eerst de topologie."
  exit 1
fi

# Opties detecteren: poortnamen afdwingen en sorteren op prioriteit (hoogste eerst).
OPTS=""
sudo ovs-ofctl --names -O OpenFlow13 dump-flows a_f1 >/dev/null 2>&1 && OPTS="--names"
SORT=""
sudo ovs-ofctl $OPTS --rsort=priority -O OpenFlow13 dump-flows a_f1 >/dev/null 2>&1 && SORT="--rsort=priority"
if sudo ovs-ofctl $OPTS -O OpenFlow13 dump-flows a_f1 | grep -q 'in_port="a_f1-eth'; then NAMED=1; else NAMED=0;
fi

# MAC van a1_gst en pid van natgw opzoeken.
mac_of() {
  local pid
```

```
  pid=$(pgrep -f "mininet:$1\$" | head -n1)
  [ -n "$pid" ] && sudo mnexec -a "$pid" ip -o link show dev "$1-eth0" 2>/dev/null \
    | awk '{for(i=1;i<=NF;i++) if($i=="link/ether") print $(i+1)}'
}
GST_MAC=$(mac_of a1_gst); GST_MAC=${GST_MAC:-00:00:00:00:00:02}
NATPID=$(pgrep -f 'mininet:natgw$' | head -n1)

P() {  # $1 = poortnaam, $2 = poortnummer: patroon dat alleen die poort selecteert
  if [ "$NAMED" = "1" ]; then echo "in_port=\"$1\"([, ]|\$)"; else echo "in_port=$2([, ]|\$)"; fi
}

tidy() {  # cookie, duration en timeouts weglaten voor leesbaarheid
  if [ "${FULL:-0}" = "1" ]; then cat
  else sed -E 's/cookie=0x[0-9a-f]+, //; s/duration=[0-9.]+s, //; s/(hard|idle)_timeout=[0-9]+, //'
  fi
}

banner() {  # $1 figuur, $2 sectie, $3 wat je moet zien, $4 onderschrift
  local line="================================================================================"
  clear
  echo "$line"
  echo " FIGUUR $1   |   $2"
  echo "$line"
  echo " Wat je moet zien : $3"
  echo " Onderschrift     : $4"
  echo "--------------------------------------------------------------------------------"
}

flows() {  # $1 switch, $2 tabel (of ""), $3 filterpatroon (of "")
  local sw="$1" tbl="$2" pat="$3" shown out
  shown="ovs-ofctl $OPTS $SORT -O OpenFlow13 dump-flows $sw $tbl"
  [ -n "$pat" ] && shown="$shown | grep -E '$pat'"
  echo "\$ $shown"
  if [ -n "$pat" ]; then
    out=$(sudo ovs-ofctl $OPTS $SORT -O OpenFlow13 dump-flows "$sw" $tbl | grep -E "$pat" | tidy)
  else
    out=$(sudo ovs-ofctl $OPTS $SORT -O OpenFlow13 dump-flows "$sw" $tbl | tidy)
  fi
  if [ -z "$out" ]; then echo "(geen regels gevonden: voer eerst genereer_verkeer.cmd uit)"; else echo "$out"; fi
  echo
}

ng() {  # commando uitvoeren op natgw
  if [ -n "$NATPID" ] && command -v mnexec >/dev/null 2>&1; then
    sudo mnexec -a "$NATPID" "$@"
  else
    echo "(natgw niet bereikbaar vanuit dit venster: typ in mininet> het commando na de \$)"
  fi
  echo
}

pause() { echo; read -r -p "[Enter = volgende figuur, q = stoppen] " k; [ "$k" = "q" ] && exit 0; }

fig0() {
  banner 0 "Overzicht: verdeling over tabellen" \
    "per switch het aantal flow entries per tabel (cores 4 tabellen, verdiepingen 5 tabellen)" \
    "Verdeling van de flow entries over de tabellen per switch."
  echo "\$ ovs-ofctl -O OpenFlow13 dump-flows <switch> | grep -o 'table=[0-9]*' | sort | uniq -c"
  for s in $SW_LIST; do
    printf "%-7s" "$s"
    sudo ovs-ofctl $OPTS -O OpenFlow13 dump-flows "$s" | grep -o 'table=[0-9]*' | sort | uniq -c \
      | awk '{printf "%s: %s   ", $2, $1} END {print ""}'
  done
}

fig1() {
  banner 1 "Sectie 1: VLAN-segmentatie en trunking" \
    "poort 2, 3, 4: push_vlan + set_field 4196, 4296, 4396; poort 1 (trunk): dl_vlan 100, 200, 300; priority=0
drop" \
```

```
    "Tabel 1 van a_f1: een hostpoort geeft ongetagd verkeer het VLAN van de poort, de trunk accepteert alleen VLAN
100, 200 en 300."
  flows a_f1 table=1
}

fig2() {
  banner 2 "Sectie 4: Gastisolatie" \
    "arp en icmp6 (135, 136) toegestaan; drops naar 10.0.200.0/24, 10.0.100.0/23, 10.0.30.0/24 en IPv6-prefixen;
meter:1 naar gateway-MAC; catch-all drop; n_packets > 0" \
    "Tabel 0 van a_f1, gastpoort, gesorteerd op prioriteit: de drops staan boven de toelating naar het gateway-
MAC."
  flows a_f1 table=0 "$(P a_f1-eth3 3)"
}

fig3() {
  banner 3 "Sectie 5: Rate limiting" \
    "meter 1 met rate=1000 burst_size=100; drie gastregels met actions=meter:1" \
    "Meter 1 (1 Mbit/s, drop-band) en de flow entries in tabel 0 die ernaar verwijzen."
  echo "\$ ovs-ofctl -O OpenFlow13 dump-meters a_f1"
  sudo ovs-ofctl -O OpenFlow13 dump-meters a_f1
  echo
  flows a_f1 table=0 "meter:1"
}

fig4() {
  banner 4 "Sectie 5: Rate limiting" \
    "packet_in_count en byte_in_count; bij bands de packet_count (gedropte pakketten) groter dan 0" \
    "Statistiek van meter 1: pakketten in de drop-band bewijzen dat de limiet is toegepast."
  echo "\$ ovs-ofctl -O OpenFlow13 meter-stats a_f1"
  sudo ovs-ofctl -O OpenFlow13 meter-stats a_f1
}

fig5() {
  banner 5 "Sectie 6: Port security" \
    "allow op dl_src=02:00:00:00:30:11 (prioriteit 20480) en daaronder een drop (20479) met n_packets > 0" \
    "Tabel 0 van a_f1, managementpoort: alleen het geregistreerde MAC-adres mag door, al het andere wordt
gedropt."
  flows a_f1 table=0 "$(P a_f1-eth4 4)"
}

fig6() {
  banner 6 "Sectie 2: MAC-learning en forwarding" \
    "bovenaan beveiligingsregels (drop dl_src ff:ff.., 0e:00.., 0x9000); geleerde bron-MAC's (priority 8191);
onderaan CONTROLLER:96" \
    "Tabel 2 van a_f1: Faucet leert bron-MAC's per poort en VLAN; onbekende bron-MAC's gaan naar de controller."
  flows a_f1 table=2
}

fig7() {
  banner 7 "Sectie 2: MAC-learning en forwarding" \
    "pop_vlan + output naar hostpoorten; output zonder pop_vlan naar trunk (poort 1); priority=0 goto_table:4" \
    "Tabel 3 van a_f1: doorsturen op doel-MAC. Hostpoorten krijgen ongetagd verkeer, de trunk behoudt de tag."
  flows a_f1 table=3
}

fig8() {
  banner 8 "Sectie 3: Flooding, ARP en IPv6 ND" \
    "broadcast (ff:ff..) en IPv6-multicast (33:33..) per VLAN naar trunk en hostpoort; drops voor 01:00:0c.. en
01:80:c2.." \
    "Tabel 4 van a_f1: broadcast en multicast blijven binnen het eigen VLAN; beheerverkeer van andere apparatuur
wordt gedropt."
  flows a_f1 table=4
}

fig9() {
  banner 9 "Sectie 7: Verkeer tussen verdiepingen, gebouwen en naar de gateway" \
    "tabel 0: dl_vlan 100, 200, 300 per poort; tabel 2: dl_dst=02:00:00:00:fe:01 -> poort 4 (natgw), hosts in
gebouw B -> poort 3 (darkfiber)" \
```

```
    "a_core heeft geen ACL's: tabel 0 = VLAN, 1 = bron-MAC, 2 = doel-MAC, 3 = flooding. Alle uitgangen zijn
trunks, dus de tag blijft."
  flows a_core table=0
  flows a_core table=2
}

fig10() {
  banner 10 "Sectie 7: Darkfiber" \
    "in_port 4 (darkfiber) met dl_vlan 100, 200 en 300 -> goto_table:1; priority=0 drop" \
    "Tabel 0 van b_core: de darkfiber vervoert alle drie de VLAN's getagd."
  flows b_core table=0
}

fig11() {
  banner 11 "Sectie 8: Stateful firewall (IPv4)" \
    "policy DROP met teller; ctstate INVALID drop; ESTABLISHED,RELATED accept met hoge teller; ctstate NEW alleen
van binnen naar buiten" \
    "Firewall op natgw: alleen intern geinitieerde verbindingen komen door, terugverkeer volgt via ESTABLISHED."
  echo "\$ natgw iptables -L FORWARD -n -v"
  ng iptables -L FORWARD -n -v
}

fig12() {
  banner 12 "Sectie 8: Stateful firewall (IPv6)" \
    "dezelfde opbouw als IPv4: policy DROP met teller, ESTABLISHED,RELATED, ctstate NEW van binnen naar buiten" \
    "De IPv6-firewall op natgw: geen NAT, maar dezelfde stateful regels."
  echo "\$ natgw ip6tables -L FORWARD -n -v"
  ng ip6tables -L FORWARD -n -v
}

fig13() {
  banner 13 "Sectie 8: NAT" \
    "SNAT-regel van 10.0.0.0/8 naar 198.51.100.2 met pkts en bytes" \
    "PAT op natgw: alle interne IPv4-hosts delen het publieke adres 198.51.100.2."
  echo "\$ natgw iptables -t nat -L POSTROUTING -n -v"
  ng iptables -t nat -L POSTROUTING -n -v
}

fig14() {
  banner 14 "Sectie 9: Pad van een pakket, gast naar ISP (a_f1)" \
    "stap 1 tabel 0 meter:1; stap 2 tabel 1 push_vlan 4296; stap 3 tabel 2 geleerd MAC; stap 4 tabel 3 output naar
trunk; extra: gast naar gast = drop" \
    "Het pad van een gastpakket door de tabellen van a_f1, en het geblokkeerde pad naar een andere gast."
  echo "# Stap 1 (tabel 0): toegestaan naar het gateway-MAC, via de meter"
  flows a_f1 table=0 "$(P a_f1-eth3 3).*dl_dst=02:00:00:00:fe:01"
  echo "# Stap 2 (tabel 1): ongetagd verkeer krijgt VLAN 200"
  flows a_f1 table=1 "$(P a_f1-eth3 3)"
  echo "# Stap 3 (tabel 2): bron-MAC van a1_gst is geleerd"
  flows a_f1 table=2 "$(P a_f1-eth3 3).*dl_src=$GST_MAC"
  echo "# Stap 4 (tabel 3): doorsturen naar de gateway via de trunk"
  flows a_f1 table=3 "dl_vlan=200,dl_dst=02:00:00:00:fe:01"
  echo "# Extra: gast naar een andere gast wordt in tabel 0 gedropt"
  flows a_f1 table=0 "$(P a_f1-eth3 3).*nw_dst=10.0.200.0/24"
}

fig15() {
  banner 15 "Sectie 9: Pad van een pakket, gast naar ISP (a_core)" \
    "stap 5 tabel 0 in_port 1 + dl_vlan=200; stap 6 tabel 2 naar poort 4 (natgw); terugweg: dl_dst a1_gst -> poort
1, daarna pop_vlan op a_f1" \
    "Het pad op de core en de terugweg naar de gast."
  echo "# Stap 5 (a_core, tabel 0): komt binnen van a_f1 met VLAN 200"
  flows a_core table=0 "$(P a_core-eth1 1).*dl_vlan=200"
  echo "# Stap 6 (a_core, tabel 2): naar natgw (poort 4)"
  flows a_core table=2 "dl_vlan=200,dl_dst=02:00:00:00:fe:01"
  echo "# Terugweg (a_core, tabel 2): naar a1_gst via poort 1"
  flows a_core table=2 "dl_vlan=200,dl_dst=$GST_MAC"
  echo "# Terugweg (a_f1, tabel 3): tag eraf, uit via de gastpoort"
  flows a_f1 table=3 "dl_vlan=200,dl_dst=$GST_MAC"
}
```

```
if [ $# -eq 0 ]; then set -- 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15; fi
total=$#; i=0
for n in "$@"; do
  i=$((i+1))
  if declare -f "fig$n" >/dev/null; then "fig$n"; else echo "Onbekend figuur: $n (zie: bash show_flows.sh lijst)";
continue; fi
  [ "$i" -lt "$total" ] && pause
done
mkdir -p ~/sdn-opdracht2/tests

cat > ~/sdn-opdracht2/tests/scale_test.sh <<'SCRIPT'

#!/bin/bash

# Gebruik: scale_test.sh [aantal] [interface] [doel] [seconden]

COUNT=${1:-500}

IFACE=${2:-a1_emp-eth0}

TARGET=${3:-198.51.100.1}

SECONDS_RUN=${4:-20}

DIR=/tmp/scale

rm -rf "$DIR"; mkdir -p "$DIR"


python3 -c '

import ipaddress, sys

count = int(sys.argv[1])

skip = {"10.0.100.11", "10.0.100.12", "10.0.100.21",

     "10.0.100.22", "10.0.100.23", "10.0.100.254"}

net = ipaddress.ip_network("10.0.100.0/23")

ips = [str(h) for h in net.hosts() if str(h) not in skip][:count]

print("\n".join(ips))

' "$COUNT" > "$DIR/ips.txt"


while read -r ip; do

  ip addr add "$ip/23" dev "$IFACE"

done < "$DIR/ips.txt"


echo "Aliassen toegevoegd: $(wc -l < "$DIR/ips.txt")"

while read -r ip; do

  ping -c "$SECONDS_RUN" -i 1 -q -I "$ip" "$TARGET" > "$DIR/ping_$ip.txt" 2>&1 &
```

Bijlage 3 Verantwoording gebruik van AI

Tool en keuze. Ik heb Claude (Anthropic) gebruikt, omdat ik daarmee tijdens het practicum kon doorvragen over mijn eigen Mininet-, Faucet- en firewallconfiguratie.

Werkwijze. Ik gebruikte de AI voor het opzetten van de structuur en de kopjes, het schrijven van conceptteksten. Verder is het gebruikt als hulp middel bij het schrijven van hulpscripts voor tests scale_test.sh en show_flows.sh en het nalezen op fouten. Ik vroeg steeds om uitleg bij mijn eigen resultaten en testoutput. Antwoorden heb ik gecontroleerd door de commando's zelf uit te voeren en de tellers, dumps en screenshots te vergelijken met de tekst.

Kritische reflectie. De AI maakte het schrijven en structureren sneller, en ik begreep door de uitleg beter hoe de OpenFlow-tabellen en de firewall samenwerken. Het risico was dat tekst of tellers niet overeenkwamen met mijn werkelijke resultaten. Daarom heb ik de tekst aangepast op mijn eigen dumps en testuitvoer. De configuratie (Faucet-YAML, Mininet-topologie, firewall- en NAT-regels) en de testscripts zijn met hulp van de AI opgebouwd en verbeterd. Ik heb de omgeving zelf draaiend gemaakt en gehouden, de tests uitgevoerd, de screenshots gemaakt, fouten opgespoord en de resultaten gecontroleerd.